zkSketch: Verifiable and Privacy-Preserving Sketch-Based Analytics via Non-Interactive Zero-Knowledge Proofs

Anonymous Author(s)

Abstract

Regulated analytical workloads — periodic financial reporting, blockchain analytics, telemetry archives — publish aggregate answers (top-k, distinct counts, frequencies, quantiles) over privately committed data streams. Approximate sketches (Count-Min, HyperLogLog, SpaceSaving, Greenwald–Khanna) are the standard plaintext tool, and non-interactive zero-knowledge proofs over commitments are the natural cryptographic primitive for verifying answers without disclosing inputs. The two combine poorly under streaming workloads, because regenerating a ZK proof over the entire historical stream after every new committed batch defeats the central appeal of sketches: incremental maintenance.

We present zkSketch, the first non-interactive ZK framework for *streaming sketch maintenance*. zkSketch maintains a privately committed sketch alongside an efficiently updatable proof object: each new batch advances the committed state under a self-contained *transition proof* local to the batch and the previous committed state; per-epoch proofs are composed online via recursive-SNARK composition into a single history root checkable in $O(1)$ work, under three composition topologies (chained / folding, binary transition tree, k-ary batched tree) with explicit trade-offs against batch size, parallelism, and per-core memory; and each transition circuit certifies the batch's *net effect* on the committed sketch via a sortedness-and-segment-aggregation *delta certificate*, rather than replaying the sketch's plaintext random-write procedure inside the circuit. The framework instantiates on the five canonical sketch families through a seven-slot interface, and four compositional techniques (SDC, HPP, NDC, CCS) close the gap to a deployment-grade service.

Keywords

zero-knowledge proofs, approximate query processing, sketches, verifiable databases, streaming, recursive SNARKs

1 Introduction

Setting. Regulated analytical workloads publish aggregate answers over privately committed data streams. Periodic financial filings report top-k counterparties and distinct-account counts over a committed transaction log; blockchain-analytics indexes report heavy hitters and frequency aggregates over committed blocks; telemetry archives report quantiles and cardinalities over vendor-signed event streams. The raw records are sensitive and cannot be shown to third-party auditors, but the published aggregates must still be verifiable end-to-end against the committed inputs. Compact summaries — Count-Min [8], HyperLogLog [10], SpaceSaving [25], Greenwald–Khanna [15] — are the plaintext tool of choice for the aggregates, and non-interactive zero-knowledge proofs over committed data [2, 17, 22, 39] are the natural cryptographic primitive for verifying answers without disclosing the inputs.

Central challenge: incremental, not from scratch. The central challenge in verifiable sketch analytics is *not* merely to prove a single sketch computation in ZK. Real analytical streams evolve continuously: new records arrive over time, the committed sketch state changes, and queries may be issued before the stream ends. Recomputing a ZK proof over the entire historical stream after every new batch would destroy the central appeal of sketches — their incremental nature in the plaintext world. We therefore ask:

> *Can a single untrusted prover maintain a privately committed sketch together with an efficiently updatable proof object, so that each new batch only requires a local transition proof, while auditors can still verify the entire committed history succinctly at any time?*

Three technical sub-problems. This framing crystallises into three concrete sub-problems that organise the rest of the paper.

(C1) Incremental verifiability. After epoch B_t arrives, the prover should produce a proof for the *transition* $(S_{t-1}, B_t) \to S_t$, not a proof that $S_t = A_\tau(B_1, \ldots, B_t)$ rebuilt from the empty initial state. Update-local proving means the per-batch prover cost depends mainly on the new batch and the changed sketch cells, not on the full history length.

(C2) Succinct history binding. Even with per-batch transition proofs $\pi_1, \ldots, \pi_t$ in hand, a verifier should not be required to check t proofs every time it audits. The system needs an updatable history accumulator that absorbs each new π_t and exposes one compact root commitment $R^{(t)}$ checkable in work independent of t. Recursive SNARKs, folding accumulators, and proof trees are the candidate primitives.

(C3) Efficient transition certification. The per-batch transition circuit must itself be cheap. A direct in-circuit replay of the plaintext sketch update procedure is expensive: sketch updates evaluate data-dependent hashes, write to private addresses determined by those hashes, handle within-batch collisions, and rebind a committed state after each write — the worst-case operation pattern for PLONKish backends. The transition circuit must therefore avoid replaying the procedure and instead certify only the batch's *net effect* on the committed state.

Two further concerns shape the same design space and are addressed jointly throughout the paper: *(C4) version-correct query answering* — a query proof must be tied to the committed sketch version containing exactly the uploaded prefix visible before the query — and *(C5) leakage control* — public artefacts such as the circuit shape and the committed capacities must not reveal private distributional information such as touched-cell counts or key skew.

Our approach. zkSketch realises the framing above in three layers.

Per-epoch transition proofs (§4, addressing C1 and C3) certify how one committed batch advances the committed sketch state.

The transition circuit verifies a *delta certificate* — a sorted, segment-aggregated patch over only the cells the batch actually changes — instead of replaying the sketch's hash-determined random writes. The certificate's shape depends only on public capacities, so the circuit is data-oblivious.

Recursive history composition (§4.3, addressing C2) absorbs each new per-epoch proof into a single history root commitment $R^{(t)}$. The root proof $\Pi^{(t)}$ is verifiable in $O(1)$ work irrespective of the number of epochs. We develop the composition design space as three topologies — chained / folding for line-rate ingest, binary transition tree for parallel audit-window provers, and k-ary batched tree for throughput-oriented deployments — and characterise their trade-offs against per-batch latency, prover memory, parallelism, and per-core budget.

Sketch instantiations and deployment-grade mechanisms (§5–§??, addressing C3, C4, and C5) instantiate the framework on the four sketch families through a seven-slot interface (Π_τ) and a two-pattern dichotomy (Pattern A linear / Pattern A max / Pattern B), tie every query proof to a committed sketch version via epoch-closure boundaries, and add four compositional techniques — shared delta codomain (SDC), hash-profile pre-commitment (HPP), negative-delta certificates (NDC), and the capacity co-sketch (CCS) — that amortise sort and hash work across coupled sketches and batches, extend Pattern A linear sketches to delete-aware updates, and bound capacity leakage under a differentially private mechanism.

Contributions.

- **Streaming sketch maintenance framework.** A committed sketch object that advances version-by-version under self-contained per-epoch transition proofs and an updatable history root, bound via recursive-SNARK composition into one root commitment $R^{(t)}$ summarising the verifiable streaming history. Merge proofs for independently maintained shards are a separate first-class operation.
- **Composition design space.** Three composition topologies — chained / folding, binary transition tree, k-ary batched tree — expose one $O(1)$ verifier interface but differ on online latency, per-batch maintenance cost, parallelism, and per-core memory; we characterise their regimes against deployment knobs.
- **Per-batch transition circuits across four sketch families.** A seven-slot interface and a two-pattern dichotomy instantiate Count-Min, HyperLogLog, SpaceSaving, and Greenwald–Khanna under one circuit pipeline organised around a verification-over-computation discipline.
- **Deployment-grade system mechanisms.** Four compositional techniques (SDC, HPP, NDC, CCS), a layered architecture with a pluggable certificate-shape catalog, a deployment-aware compiler over a three-dimensional cost surface (prover × verifier × leakage), and workflow decomposition turn the per-circuit baseline into a deployable service.

Scope. ZK certifies exact adherence to the *declared sketch algorithm*, not exact equality with the true answer over the underlying dataset; approximation error is inherent to sketch theory. Input completeness — that every real-world event reached the sketch — requires an authenticated input log on the data-owner side and is outside the cryptographic scope of zkSketch.

Roadmap. §2 fixes preliminaries. §3 defines the system model, threat model, and committed sketch object. §4 develops the streaming sketch maintenance framework: per-epoch transition proofs, the long-history bottleneck, recursive interval proofs, and three composition topologies. §5 instantiates the framework circuit-by-circuit on Count-Min and Greenwald–Khanna as the two pattern representatives; HyperLogLog and SpaceSaving appear in the appendix. §?? addresses deployment-grade system mechanisms. §6 describes the planned evaluation, §7 discusses related work, and §?? concludes.

2 Preliminaries

2.1 Zero-Knowledge Proofs (ZKPs)

Zero-knowledge proofs (ZKPs) allow a prover to convince a verifier that a public statement is true without revealing any information beyond the validity of the statement itself. ZKP protocols are commonly classified as either interactive [14] or non-interactive [5]. For certain public-coin interactive protocols [13], the Fiat–Shamir transformation [9] can be used to turn interactive ZKPs into non-interactive ZKPs. In this work, we focus on non-interactive ZKPs, where the proof is consolidated into a single message that can be verified later by any verifier. This makes such proofs particularly suitable for asynchronous settings in which the prover and verifier need not be online at the same time. More specifically, we use *zero-knowledge succinct non-interactive arguments of knowledge* (ZK-SNARKs) [5], which produce short proofs and support fast verification properties that are well suited to the offline-checkable model. A ZK-SNARK provides three standard guarantees:

- *Completeness*: an honest prover can convince an honest verifier of any true statement.
- *Knowledge soundness*: a dishonest prover cannot convince a verifier of a false statement except with negligible probability, and an extractor recovers the witness from any successful prover.
- *Zero-knowledge*: the verifier learns nothing about the witness beyond the validity of the statement.

At a high level, a ZK-SNARK proves knowledge of a private witness that satisfies a publicly specified relation for a given public statement. The witness contains the hidden information needed to justify why the statement is true; depending on the application, it may include secret inputs, intermediate values, or an execution trace of the computation being certified. To enable efficient proving and verification, the target computation is encoded as an arithmetic circuit, or equivalently as a system of algebraic constraints over a finite field. This circuit is public and specifies how the public inputs and private witness must relate. In practice, the computation is represented by constraints, often expressed as low-degree polynomial equations, that are satisfied exactly when the witness corresponds to a valid execution of the computation. The prover then generates a succinct proof that it knows a witness satisfying all of these constraints, without revealing the witness itself.

Shall we introduce ZKSNARK and ZKSTARK here?

3 System Model and Problem Statement

This section defines the contract that the rest of the paper implements. The central object is a versioned, privately committed sketch

over a stream of uploaded records. The prover maintains the sketch and its proof history; the verifier checks published answers against public commitments without learning the records or the private sketch state.

3.1 Streaming Workload and Versions

The logical input is an interleaving of uploads and queries:

$$\mathcal{X} = \big((o_1, x_1), (o_2, x_2), \ldots\big), \qquad o_j \in \{\text{upload}, \text{query}\}.$$

If $o_j = \text{upload}$, then x_j is a private record. If $o_j = \text{query}$, then $x_j = q$ is a public query parameter, such as a key for point-frequency estimation, a rank value for quantile queries, or $\perp$ for parameter-free queries such as cardinality.

Let

$$u_1, u_2, \ldots$$

denote the subsequence of uploaded records in their logical order. For proof generation, zkSketch groups consecutive uploads into epochs

$$B_t = (u_{\ell_{t-1}+1}, \ldots, u_{\ell_t}), \qquad 0 = \ell_0 < \ell_1 < \cdots .$$

Epoch boundaries are a systems knob: they may be chosen by record count, wall-clock time, memory pressure, or query arrival. The special case $|B_t| = 1$ gives item-wise proofs; larger epochs amortize commitment openings, sorting, hash checks, aggregation, and proof generation.

For a sketch family τ, let A_τ be the declared sketch algorithm. Starting from the public empty state S_0, the versioned state after epoch t is

$$S_t = A_\tau.\mathsf{UpdateEpoch}(S_{t-1}, B_t),$$

where $A_\tau.\mathsf{UpdateEpoch}$ denotes applying the ordinary item-update rule of A_τ to all records in B_t, in order.

Queries are interpreted over a closed prefix of the stream. If a query arrives while an epoch is open, the prover first closes that epoch, commits to it, proves the corresponding transition, and only then emits the query proof. Thus every query is tied to a unique sketch version. For a query at logical time j, let

$$\mathrm{ver}(j)$$

be the smallest closed version whose epochs contain exactly all uploads that occur before j. The intended answer is

$$ans = A_\tau.\mathsf{Decode}_{x_j}\big(S_{\mathrm{ver}(j)}\big).$$

This version-closure rule is the only place where query timing enters the correctness definition.

3.2 Committed Sketch Object and Public Log

For each sketch instance, zkSketch maintains a committed object

$$\mathrm{Obj}_t = (id, \tau, params, \mathsf{M}, t, \mathsf{Com}(S_t), R^{(t)}, policy).$$

Here id identifies the instance, τ identifies the sketch family, and $params$ contains public accuracy and capacity parameters such as the Count-Min dimensions (d, w), the HyperLogLog register count, the SpaceSaving capacity, or the GK capacity parameter. The metadata M fixes all public algorithm choices required to check proofs: hash profile, counter representation, commitment scheme, circuit capacities, and proof-system parameters.

Figure placeholder. A left-to-right pipeline. Left: an operation stream with uploads and queries. Middle-left: consecutive uploads are sealed into epochs $B_1, B_2, \ldots$, each with a public commitment C_{B_t}. Middle-right: an untrusted prover maintains $\mathsf{Com}(S_t)$ and $R^{(t)}$. Each epoch emits a transition proof; queries emit decode and optional release proofs against a closed version. Right: an offline verifier checks the public log: epoch commitments, state commitments, history roots, metadata, answers, and proofs.

Figure 1: System model of zkSketch. Uploads are grouped into committed epochs, each epoch advances a committed sketch state, and queries are answered only against closed versions. The verifier checks public commitments, roots, answers, and proofs offline.

The state commitment $\mathsf{Com}(S_t)$ binds the hidden sketch state at version t. The history root $R^{(t)}$ binds the ordered sequence of committed epochs and state transitions up to version t. The two commitments play different roles: $\mathsf{Com}(S_t)$ says what the current private sketch state is, while $R^{(t)}$ says why that state is the result of the committed stream prefix. Section 4 constructs $R^{(t)}$ using recursive history composition.

The public log at version t contains the artifacts needed for offline verification:

$$\mathsf{Log}^{(t)} = \big\{C_{B_i},\ \mathsf{Com}(S_i),\ R^{(i)},\ \pi_i\big\}_{i=1}^{t} \cup \mathsf{QueryLog}^{(t)}.$$

Here C_{B_i} is a binding commitment to epoch B_i, $\mathsf{Com}(S_i)$ is the post-epoch sketch-state commitment, $R^{(i)}$ is the history root, and π_i is the transition proof for epoch i. The query log stores decoded answers together with the corresponding version number and decode/release proofs.

The public release policy *policy* specifies deterministic post-processing of decoded answers, such as rounding, suppression below a threshold, or noise generation from a committed seed. Since the policy is public, a release proof can certify that the published answer was derived correctly from the decoded sketch answer.

3.3 Actors and Threat Model

There are three roles.

Data owners. Data owners generate upload records and publish binding commitments to records or epochs before those records are used in sketch maintenance. After committing, data owners need not participate in proof generation.

Prover. The prover maintains the private sketch state and produces all non-interactive proofs. The prover is untrusted. It may try to omit a committed upload, reorder epochs, use undeclared sketch parameters or hash functions, fabricate a state transition, answer from a stale version, or publish an answer that is not the declared decoder's output.

Verifier. The verifier is an auditor, regulator, client, or third party who checks the public log offline. The verifier sees commitments, metadata, version numbers, history roots, answers, and proofs, but

not raw records or private sketch states. The verifier may be curious; proof artifacts should therefore reveal no private information beyond what is implied by public metadata, public capacities, query timing, and released answers.

We assume the underlying NIZK is complete, knowledge-sound, and zero-knowledge; commitments are computationally binding and hiding when required; and the hash function used to bind ordered histories is collision-resistant.

3.4 Problem Statement

For a closed version t, the intended sketch state is the unique state reachable from S_0 by applying the declared epoch updates $B_1, \ldots, B_t$ in order. We write this state as S_t, with the per-epoch invariant

$$S_i = A_\tau.\mathsf{UpdateEpoch}(S_{i-1}, B_i).$$

The public commitments $\mathsf{Com}(S_t)$ and $R^{(t)}$ are required to bind this state and its ordered update history. For any query q answered at closed version t, the intended answer is

$$ans = A_\tau.\mathsf{Decode}_q(S_t).$$

The goal of zkSketch is to construct non-interactive proofs for committed sketch maintenance satisfying the following properties.

Update soundness. If an epoch-transition proof verifies for public input

$$(\mathsf{Com}(S_{t-1}), C_{B_t}, \mathsf{Com}(S_t), \mathsf{M}),$$

then $\mathsf{Com}(S_t)$ opens to the state obtained by applying the declared epoch update to an opening of $\mathsf{Com}(S_{t-1})$ and an opening of C_{B_t}. A malicious prover cannot fabricate the transition, alter the epoch, or use different metadata except with negligible probability.

History soundness. If the root proof for $R^{(t)}$ verifies, then the current state commitment $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying the ordered committed epochs $B_1, \ldots, B_t$. Dropping, replacing, or reordering an epoch changes the history root except with negligible probability.

Query-prefix soundness. If a decode proof verifies for a query at logical time j, then the answer is the declared decoder's output on the closed sketch version containing exactly all uploads before j. In particular, the prover cannot answer from a stale version or from a state containing uploads that should not yet be visible to the query.

Zero knowledge and circuit obliviousness. Proofs hide private witnesses, including raw records, sketch states, certificate tables, commitment openings, and intermediate values. In addition, the public circuit shape depends only on declared public capacities and metadata, not on private quantities such as the number of distinct keys, touched cells, fusions, or heavy hitters. If capacities are selected adaptively from data, they must be released through the capacity mechanism of Section **??**.

Efficiency objective. The prover should maintain the sketch incrementally: the cost of proving epoch B_t should depend primarily on B_t, the sketch parameters, and the certified batch effect, not on the full history length. The verifier should check a query using a constant number of succinct proofs, rather than scanning all past epochs or all raw records.

Non-goals. zkSketch certifies faithful execution of the declared sketch algorithm over the committed input stream. It does not certify that the approximate sketch answer equals the exact answer over the raw dataset. Approximation error is inherited from the chosen sketch family and its public parameters. Input completeness—that every real-world event was actually committed into the stream—requires an authenticated input log or data-owner-side mechanism and is outside the cryptographic scope of zkSketch.

4 zkSketch Framework: Verifiable Streaming Sketch Maintenance

We introduce the framework of zkSketch in this section. In the streaming data setting, data updates arrive continuously while the prover operates under bounded memory and computation. The prover therefore cannot wait until the entire stream has been observed before producing a ZKP: the stream is potentially unbounded, and any client query issued in the meantime would have to wait. Our framework adopts the *incremental update* idea: the prover emits proofs as updates arrive, so that when new updates come in, proof generation builds on the proofs already established rather than starting from scratch. This significantly reduces the latency a client sees when querying the data statistics based on the current sketch state, since at any moment the prover already holds an up-to-date proof of all updates processed so far.

In the following, Section 4.1 introduces the most direct realisation (one ZKP per update) and exposes the verifier-side $\Theta(E)$ bottleneck it leaves open. Section 4.2 collapses the chain of per-update proofs into a single root proof via recursive interval proofs, fixing the merge atom and the interval-proof contract. Section 4.3 schedules these merges over a streaming history as a forest of perfect subtrees, structurally analogous to a Merkle Mountain Range [35] but used only to organise the recursive composition. Section 4.4 handles queries interleaved with updates. The end-to-end soundness and privacy guarantees of the framework are stated in Appendix A.

4.1 Per-update proof chains and their cost

To process the stream, the most direct strategy is to generate one ZK proof per update. Whenever a new update b_i arrives, the prover produces a proof π_i certifying that the committed sketch state was advanced by exactly b_i.

We formalize one such proof as a per-update transition proof. Each π_i has public input

$$x_i \;=\; \big(\mathsf{Com}(S_{i-1}),\; C_{b_i},\; \mathsf{Com}(S_i),\; \mathsf{M}\big),$$

where M is the public metadata that fixes the sketch parameters, hash profile, counter representation, commitment scheme, and circuit capacities. The private witness w_i contains the old state, the opening of b_i, the new state, and any auxiliary witness required by the update circuit of Section 5. The proof certifies membership in the update relation

$$\mathcal{R}^{\tau}_{\mathsf{upd}} \;=\; \{(x_i, w_i) \;:\; S_i = A_\tau.\mathsf{Update}(S_{i-1},\, b_i)\}\,,$$

where τ identifies the sketch family.

The proof π_i is self-contained: it verifies against x_i alone, without reading past or future updates, so the prover can emit $\mathsf{Com}(S_i)$ and π_i online as soon as b_i is processed. Consecutive proofs are chained through their boundary state commitments: the next proof

π_{i+1} takes $\mathsf{Com}(S_i)$ as its previous-state commitment. We call the resulting sequence the *per-update proof chain.*

A verifier that wishes to check all updates have been correctly applied to the sketch verifies every proof in the chain and confirms that adjacent proofs agree on their boundary commitment: the output state commitment of π_i must equal the input state commitment of π_{i+1}. Provided the chain is intact and every π_i verifies, the verifier learns that the final committed state is the result of applying all committed updates in order.

The cost of this scheme falls on the verifier. After E updates have been processed, the verifier has to check E proofs and verify $E-1$ boundary equalities between consecutive proofs. Verification time scales as $\Theta(E)$ in the number of updates. For sketches maintained over long data streams, the proof chain grows with the stream length, eventually exceeding what a bounded verifier can reasonably ingest and verify end-to-end.

A natural attempt to escape this bottleneck is to enlarge each update so that fewer of them are needed. Bounded prover memory and verifier compute prevent this: a single ZKP over a very large update becomes prohibitively expensive to generate and to verify, and the deployment's per-update latency budget caps the update size in practice. The per-update strategy therefore cannot escape the $\Theta(E)$ verifier cost simply by growing each update. Local soundness of individual transition proofs is useful, but it is not enough for long streams; Section 4.2 collapses the chain into a single root proof via recursive composition.

4.2 Collapsing the chain via recursive composition

Why we need a stronger primitive. The per-update chain of Section 4.1 leaves the verifier with $\Theta(E)$ proofs to check. The chain links those proofs only structurally — boundary-state commitments tie each π_i to its neighbours, but no proof in the chain ever *verifies* any of the others, so checking one proof tells the verifier nothing about the rest. What we need is a primitive that lets a single most-recent proof subsume all earlier proofs, so the verifier checks one object regardless of how long the stream is.

Recursive proof composition. A zero-knowledge proof system (§2.1) reduces a computation to an arithmetic circuit and emits a certificate that the circuit accepted its inputs. The system's *verifier* is itself a deterministic algorithm; nothing stops us from encoding that verifier as a circuit and asking a fresh proof to certify a computation that *contains* the verifier as a subroutine. In this framework, we use the recursive proof composition [16, 32] to achieve this. A new circuit first runs the ZKP verifier on a previously emitted proof and then runs its native computation (i.e. the updates on the sketches), then emits a single new ZKP that certifies both at once. The new proof is a transitive certificate for everything the old proof certified, plus the new local step.

The database analogue is an incrementally maintained materialized view. Each maintenance step normally carries a local *provenance certificate* attesting "this step is correct given the previous view state." Plain provenance is local: it does not retroactively certify earlier updates, so an auditor wanting to trust the view today still has to check the provenance certificate of every past update. With recursive composition, each new provenance certificate *also* verifies the previous certificate inside the same circuit, so the most-recent certificate transitively certifies the entire maintenance history — an auditor inspects exactly one certificate to be convinced of the whole view. Applied to the per-update chain of Section 4.1: the prover emits a single $\Pi^{(E)}$ certifying that all E transitions were sound, in order, against the public stream of update commitments $C_{b_1}, \ldots, C_{b_E}$. We build the construction step by step.

The merge atom. The basic operation is a *recursive merge step* on two adjacent interval proofs $\Pi_{[i,m]}$ and $\Pi_{[m+1,j]}$ (Figure 2). The merge circuit verifies both children in-circuit, enforces that the shared boundary commitment $\mathsf{Com}(S_m)$ exposed as the left child's output matches the one exposed as the right child's input, combines the two ordered-update digests into one, and emits a single parent proof $\Pi_{[i,j]}$ on the outer boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_j))$. The intermediate $\mathsf{Com}(S_m)$ is absorbed: it no longer appears on the parent's public boundary. A verifier holding $\Pi_{[i,j]}$ checks one ZKP and learns that both children were sound *and* agreed on their join point.

Figure placeholder. One recursive merge step (the atom of the construction). *Inputs (bottom of the figure):* two child interval proofs. The left child $\Pi_{\text{left}} = \Pi_{[i,m]}$ carries the public boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_m), H_{[i,m]})$; the right child $\Pi_{\text{right}} = \Pi_{[m+1,j]}$ carries $(\mathsf{Com}(S_m), \mathsf{Com}(S_j), H_{[m+1,j]})$. Draw each child as a small box with three labelled tabs on its public boundary, and use blue for state-commitment boundaries that will survive into the parent, grey for the shared $\mathsf{Com}(S_m)$ that will be absorbed, orange for the ordered update digests H. *Centre of the figure:* the recursive merge circuit, drawn as a larger box with a small ZKP-verifier icon inside. Internally the box performs three operations: (i) it runs the ZKP verifier on each child against its own public boundary; (ii) it checks that the shared $\mathsf{Com}(S_m)$ exposed as the left child's *output* equals the one exposed as the right child's *input*, highlighted with a dashed equality bracket between the two grey tabs; (iii) it combines the two ordered update digests via $H_{[i,j]} = \mathsf{H}(i, m, j, H_{[i,m]}, H_{[m+1,j]})$, shown as an orange merge node. *Output (top of the figure):* one parent interval proof $\Pi_{\text{parent}} = \Pi_{[i,j]}$ exposing only the outer boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_j), H_{[i,j]})$; the shared $\mathsf{Com}(S_m)$ does not appear on the parent's public boundary (it has been "hidden" behind the merge). *Caption hook:* include a small inset diagram on the right suggesting that iterating this step over a forest of size-2^k subtrees yields a single root proof on demand (forward-pointer to Figure 4).

Figure 2: The atom of recursive composition. Two child interval proofs go in, one parent interval proof comes out. The merge circuit runs the ZKP verifier on each child inside a new circuit, enforces that the shared boundary state commitment $\mathsf{Com}(S_m)$ agrees on both sides, and combines the ordered update digests. Iterating this step over a forest of size-2^k subtrees (Figure 4) collapses an arbitrary-length committed history into one root proof checkable in $O(1)$ time.

The interval-proof contract. We make the construction precise by giving the merge atom a contract. For an interval $[i, j]$ of updates, an interval proof $\Pi_{[i,j]}$ certifies that the committed state at the end of the interval is reachable from the committed state before the interval:

$$\mathsf{Com}(S_{i-1}) \xrightarrow{b_i, b_{i+1}, \ldots, b_j} \mathsf{Com}(S_j).$$

Its public boundary is $\mathsf{pub}_{[i,j]} = (i,\ j,\ \mathsf{Com}(S_{i-1}),\ \mathsf{Com}(S_j),\ H_{[i,j]},\ \mathsf{M})$, where $H_{[i,j]}$ is a collision-resistant digest of the ordered update commitments $C_{b_i}, \ldots, C_{b_j}$ that binds membership and order: omitting, replacing, or reordering an update changes the public input. Two rules complete the contract. A leaf interval is a single update:

$$\Pi_{[j,j]} = \pi_j, \qquad H_{[j,j]} = \mathsf{H}(j,\ C_{b_j}).$$

A merge over any split $[i, m] \cup [m{+}1, j]$ verifies both children against their own public boundaries, checks $\mathsf{Com}(S_m)$ agrees, and combines digests as

$$H_{[i,j]} = \mathsf{H}(i,\ m,\ j,\ H_{[i,m]},\ H_{[m+1,j]}).$$

The verification target on the parent is exactly: every child verifies and every shared boundary agrees.

Running example (part I).. We thread a single running example through this subsection and Section 4.3 that grows with the stream. Suppose the first two updates b_1, b_2 have been processed, with per-update proofs π_1, π_2 binding $\mathsf{Com}(S_0) \xrightarrow{b_1} \mathsf{Com}(S_1)$ and $\mathsf{Com}(S_1) \xrightarrow{b_2} \mathsf{Com}(S_2)$. Lifting each π_j to a leaf interval gives $\Pi_{[1,1]}, \Pi_{[2,2]}$. One application of the merge atom verifies both leaves, checks that the shared $\mathsf{Com}(S_1)$ agrees, combines digests as $H_{[1,2]} = \mathsf{H}(1, 1, 2, H_{[1,1]}, H_{[2,2]})$, and emits

$$\Pi_{[1,2]} : \quad \big(1,\ 2,\ \mathsf{Com}(S_0),\ \mathsf{Com}(S_2),\ H_{[1,2]},\ \mathsf{M}\big).$$

The intermediate $\mathsf{Com}(S_1)$ has been absorbed; the parent exposes only the outer boundary. We continue this example in Section 4.3 as more updates arrive.

The chain structure: a natural single-thread organisation. The simplest way to apply the merge atom to a streaming history is to maintain one accumulating root proof $\Pi^{(t)}$, extending it by one merge each time a fresh leaf arrives: $\Pi^{(1)} = \pi_1$, and at $t \geq 2$ the prover merges $\Pi^{(t-1)}$ with $\Pi_{[t,t]} = \pi_t$ to obtain $\Pi^{(t)}$ (Figure 3a). The prover therefore always holds a current root proof; a query arriving at any moment reads $\Pi^{(t)}$ directly, so the query latency is $O(1)$ *by construction*. The online cost per update is one merge.

Folding-style realisations [32] keep that per-update cost constant in practice: instead of running the full in-circuit ZKP verifier at every step, the prover folds the new instance into an accumulator at constant cost and emits a succinct SNARK only when a verifier asks for one — the recursion depth no longer grows with the stream length. The chain has *one* structural limitation, however: each merge depends on the previous one finishing, so the schedule is purely sequential. A multi-core or GPU prover has no way to use its parallel resources across the stream, and on high-velocity streams this becomes the bottleneck.

Root interface. For a stream prefix of length t, the prover publishes the pair

$$\big(R^{(t)},\ \Pi^{(t)}\big), \qquad R^{(t)} = \mathsf{H}\big(1,\ t,\ \mathsf{Com}(S_0),\ \mathsf{Com}(S_t),\ H_{[1,t]},\ \mathsf{M}\big),$$

where $\Pi^{(t)} = \Pi_{[1,t]}$ is the root interval proof. The verifier checks one succinct proof against the $O(1)$-size root commitment, regardless of t. How the prover can *parallelise* this work to keep up with high-velocity streams, trading some of the chain's $O(1)$ query latency for parallel proving, is the subject of Section 4.3.

4.3 Maintaining the root proof over a streaming history

Section 4.2 introduced the chain organisation of the merge atom: one merge per update, root always ready, $O(1)$ query latency. The chain is sequential by construction — each merge waits for the previous to finish — so on a multi-core or GPU prover it leaves parallel resources idle. High-velocity streams (financial tick data, blockchain analytics, IoT telemetry) routinely produce thousands of updates per second; a chain paying even tens of milliseconds per merge saturates well below line rate. This section asks how to schedule the merges so disjoint pieces of the history are proved concurrently. Two refinements answer that question and differ in how they treat a fixed window versus a live stream; both trade some of the chain's query latency for parallel proving work.

First parallel schedule: the balanced binary tree. When E is known in advance and a power of two, the natural parallelisation organises the E leaf proofs as a balanced binary tree of depth $\lceil \log_2 E \rceil$ (Figure 3b). Every merge in the tree applies the same fixed-shape circuit; disjoint subtrees can be built in parallel across cores; the verifier still checks one root proof. The trade-off versus the chain is concrete: appending a new leaf regenerates the $\lceil \log_2 E \rceil$ merge proofs along the spine from the new leaf to the root, so the per-update latency to refresh the root is $\Theta(\log E)$ rather than $O(1)$. *Query latency is higher than the chain's* when the prover must refresh the root on demand, but throughput on a multi-core prover is far higher. This makes the binary tree the natural choice for windowed batch processing where the prover commits one tree per window.

The streaming reality. The balanced binary tree assumes E is known in advance and is a power of two. In live streaming neither holds: E is unknown and queries arrive at unpredictable moments. We need a structure that (i) accommodates any current length t, (ii) admits cheap incremental extension as new leaves arrive, and (iii) can deliver a single root proof at any moment on demand.

Second parallel schedule: a forest of perfect subtrees. We organise the current history $[1, t]$ as a forest of perfect interval-proof subtrees whose sizes are the powers of two appearing in the binary representation of t. Writing $t = 2^{k_1} + 2^{k_2} + \cdots + 2^{k_\ell}$ with $k_1 > k_2 > \cdots > k_\ell \geq 0$, the forest holds $\ell \leq \lceil \log_2(t{+}1) \rceil$ surviving subtree roots, one of size 2^{k_r} for each set bit. For instance at $t = 11 = 8 + 2 + 1$ the forest holds three roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ (Figure 3c). The skeleton is the Bentley-Saxe static-to-dynamic transformation [4], and the layout is structurally analogous to a Merkle Mountain Range [35]; we borrow only the organisation that schedules the recursive merges, and do not derive membership proofs from the forest.

Incremental extension. When π_{t+1} arrives, the prover inserts a fresh size-1 leaf $\Pi_{[t+1,t+1]}$ on the right. If a size-1 subtree was already the rightmost surviving root, the two merge into a size-2 subtree

Figure placeholder. Three schedules for composing interval proofs over a streaming history. (a) *Linear chain*: a single accumulating interval $\Pi_{[1,t]}$ is extended by one merge per arrival; recursion depth of the root proof grows linearly with t. (b) *Balanced binary tree* (idealised, $E = 8$): leaves $\Pi_{[1,1]}, \ldots, \Pi_{[8,8]}$ at the bottom, internal merges at intervals $[1,2]$, $[3,4]$, $[5,6]$, $[7,8]$, $[1,4]$, $[5,8]$, $[1,8]$; requires E known in advance and a power of two. (c) *Streaming forest* (general t, illustrated at $t = 11 = 8 + 2 + 1$): three surviving subtree roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ of sizes 8, 2, 1. Draw the on-demand chaining as dashed merges from left to right that produce a single root $\Pi^{(11)}$ when a query or verifier request triggers finalisation. All three publish the same verifier interface $(R^{(t)}, \Pi^{(t)})$.

Figure 3: Three composition schedules for interval proofs. Panel (a) is the chain of §4.2 reproduced for comparison; panels (b) and (c) are the parallel schedules developed in this section. The chain is sequential with $O(1)$ query latency; the balanced binary tree is parallelisable but assumes a fixed, power-of-two history; the streaming forest accommodates any t and finalises a single root proof on demand.

Figure placeholder. Streaming forest after $t = 1, 2, 3, 4$ committed updates. *Panel t=1:* the forest holds a single size-1 leaf $\Pi_{[1,1]}$. *Panel t=2:* the new leaf $\Pi_{[2,2]}$ cascades with the existing size-1 root; the forest collapses to one size-2 subtree $\Pi_{[1,2]}$, with the absorbed $\mathrm{Com}(S_1)$ shown crossed out behind the merge. *Panel t=3:* the new leaf $\Pi_{[3,3]}$ is size-1 and meets a size-2 neighbour; no cascade fires. Two surviving roots $\Pi_{[1,2]}, \Pi_{[3,3]}$. *Panel t=4:* the new leaf $\Pi_{[4,4]}$ cascades twice: first with $\Pi_{[3,3]}$ into $\Pi_{[3,4]}$, then with $\Pi_{[1,2]}$ into $\Pi_{[1,4]}$. The forest collapses to a single size-4 subtree. Dashed arrows above each panel indicate the on-demand finalisation that chains surviving roots into one $\Pi^{(t)}$. Use orange for ordered-digest combinations and grey for absorbed boundary commitments.

Figure 4: Streaming forest evolving with the running example. Each panel shows the forest after one more update arrives; cascades fire whenever adjacent subtrees have equal size. At any time, on-demand finalisation (dashed) produces a single root proof $\Pi^{(t)}$ verifiable in $O(1)$ time. Under the eager peak-chain policy (§4.3), the dashed chain in each panel is what the prover maintains continuously, with the cascade at t=4 rebuilding the chain over the new single peak.

via one application of the merge atom; if a size-2 subtree was also present, the cascade continues, and so on until no two equal-sized neighbours remain. The cascade visits at most $\lceil \log_2(t{+}1) \rceil$ levels and emits $O(\log t)$ merges in the worst case, with $O(1)$ amortised over the stream by the standard binary-counter argument. Merges on disjoint subtrees run in parallel.

Producing the root on demand. At any moment the prover can compose the surviving forest roots $\Pi_{[1,2^{k_1}]}, \ldots, \Pi_{[\sum_{r<\ell} 2^{k_r}+1,\, t]}$ into a single root proof $\Pi^{(t)}$ by chaining them left-to-right with $\ell - 1 < \lceil \log_2(t{+}1) \rceil$ additional merges, and publishes $(R^{(t)}, \Pi^{(t)})$ to the verifier. This finalisation is triggered by a query or by an external verification request; it is not paid on every update. The amortised online cost per update remains $O(1)$, and the on-demand cost of delivering a root is $O(\log t)$ — one logarithmic factor higher than the chain's $O(1)$ ready root, exchanged for the across-stream parallelism the chain could not provide.

Lazy vs. eager peak chain. The on-demand finalisation just described is the *lazy* policy; an alternative *eager* policy maintains a chain over the current peaks at all times, so a query receives a ready-made root proof without further work. Both policies share $O(1)$ amortised total work per insert: lazy concentrates $O(\log t)$ at query time, while eager pays a cascade-aligned chain rebuild at insert time — one extra merge per peak consumed by the cascade. The forest skeleton is Bentley-Saxe's static-to-dynamic transformation [4], and the lazy/eager dichotomy is the standard decomposable-data-structure trade-off, formally analysed in the competitive sense by Mathieu et al. [24]. The closest existing zero-knowledge system, Mina's scan state [26], also maintains a rolling root eagerly, but uses a parallel-scan schedule over full binary trees rather than the peak-chain shape we propose here. Read against the chain of Section 4.2: the eager policy *recovers* the chain's $O(1)$ query latency by paying a chain-rebuild at cascade-aligned insert bursts, so it sits between the pure chain (low query latency, no parallelism) and the lazy forest (parallel, $O(\log t)$ query). zkSketch takes the eager peak chain as its default because streaming workloads issue queries at unpredictable times and the user-visible cost is query latency; the lazy variant remains preferable when queries are rare relative to updates.

We clarify two strategies here. If no query arrives, then we build the forest in a regular way. Otherwise, we check the number of updates we have not yet addressed. We handle those in a separate tree and then merge it with the previous chained peak proofs. In this way, we can reduce the query latency as much as possible.

Running example (part II).. Continuing the running example: after $t = 2$ a cascade merged two size-1 leaves into the size-2 subtree $\Pi_{[1,2]}$, so the forest is $\{\Pi_{[1,2]}\}$. When π_3 arrives, the new leaf $\Pi_{[3,3]}$ is size-1 and the rightmost surviving root is size-2, so no cascade fires; the forest becomes $\{\Pi_{[1,2]}, \Pi_{[3,3]}\}$. When π_4 arrives, the new leaf $\Pi_{[4,4]}$ merges with $\Pi_{[3,3]}$ into $\Pi_{[3,4]}$, which is itself size-2, so the cascade continues and merges with $\Pi_{[1,2]}$ into $\Pi_{[1,4]}$; the forest collapses to $\{\Pi_{[1,4]}\}$ (Figure 4). Under the lazy policy, on-demand finalisation at any of these moments chains the surviving roots into a single $\Pi^{(t)}$ that the verifier checks in $O(1)$ time against $R^{(t)}$; under the eager policy, the same chain is already maintained, and the cascade at t=4 rebuilds it over the new single peak.

Design-space summary. The three composition schedules trace one curve trading parallel proving work against query latency:

- *Chain* (§4.2): no parallelism across the stream, $O(1)$ query latency at every t. Best for moderate-rate streams or query-dominated SLAs on a single core.

- *Balanced binary tree*: full parallelism across cores, $\Theta(\log E)$ query latency, requires E known and a power of two. Best for windowed audits with regular window boundaries.
- *Streaming forest with eager peak chain*: full parallelism, $O(1)$ query latency under the eager policy (and $O(\log t)$ under the lazy policy), accommodates any t and online queries. The default for zkSketch.

4.4 Answering queries during streaming

Queries arrive interleaved with updates and must observe a clean prefix of the stream. The streaming forest of Section 4.3 supports this directly.

When a query arrives at logical time t, the prover first treats t as a version boundary. Any records that have arrived since the last committed update $\mathsf{Com}(S_r)$ but have not yet been folded into an update event are sealed as a new update b_{r+1}; the prover emits $\mathsf{Com}(S_{r+1})$ and the corresponding per-update proof π_{r+1}, and inserts the new leaf into the streaming forest via the cascade of Section 4.3.

The prover then triggers on-demand finalisation: chain the surviving forest roots into a single root proof $\Pi^{(t)}$ against the root commitment $R^{(t)}$. This pair certifies that $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying the committed updates $b_1, \ldots, b_t$ in order.

For the query itself, the prover emits an answer *ans* together with a *decode proof* π_q certifying

$$ans = A_\tau.\mathsf{Decode}_q(S_t),$$

where the public input of π_q binds to the $\mathsf{Com}(S_t)$ that appears on the public boundary of $\Pi^{(t)}$. The pair $(\Pi^{(t)}, \pi_q)$ therefore ties the answer to exactly the prefix of committed updates visible at time t.

5 Circuit Designs for Sketches

§4 fixed the proof-orchestration layer: per-epoch transition proofs as the basic proving unit, recursive history composition for $O(1)$-cost verification. It deferred the question this section answers: *how is one per-epoch transition proof constructed?* That is, given a committed epoch B_t, an old committed state $\mathsf{Com}(S_{t-1})$, and the published new state $\mathsf{Com}(S_t)$, what does the circuit certify and how? We build the answer step by step.

The strawman: replay the plaintext update inside the circuit. The most direct realisation is to encode the plaintext sketch update record-by-record. For each $u_i \in B_t$ the circuit (a) opens u_i from C_{B_t}, (b) evaluates the keyed hashes $\{h_g(u_i)\}_{g\in[d]}$ in-circuit, (c) opens the touched cells of the current intermediate state S_{i-1} from a commitment $\mathsf{Com}(S_{i-1})$, (d) updates those cells per the sketch's algorithm to produce S_i, and (e) rebinds the state commitment to $\mathsf{Com}(S_i)$. After $m = |B_t|$ such steps the final state S_m is checked against the published $\mathsf{Com}(S_t)$.

Why the strawman is unaffordable. The strawman pays in three structural ways. First, the *witness must materialise all m+1 intermediate sketch states* $S_0, S_1, \ldots, S_m$, because each per-record step depends on the previous one. For a Count-Min state of $|S| = dw \approx 4.6 \times 10^5$ cells (the running parameters $d = 7$, $w = 2^{16}$) and a batch of $m = 1024$, this is $\sim 4.7 \times 10^8$ field elements in the witness for a *single* epoch proof — an order of magnitude beyond the largest practical PLONKish circuit. Second, the m intermediate state commitments $\mathsf{Com}(S_0), \ldots, \mathsf{Com}(S_{m-1})$ must each be opened and rebound in-circuit, paying $\Theta(m)$ commitment-update work plus per-cell openings. Third, the keyed hash evaluations live inside the circuit per record, contributing $\Theta(mdH)$ constraints under any in-circuit hash family (Appendix B quantifies the contribution).

Why this is a different bottleneck from §4.1. §4.1 treated each update as a single proving unit and addressed the verifier-side cost of a long chain of per-update proofs. Each π_i was a black box there. The strawman above instead opens that black box: a single update is internally a group of m records that the prover has accumulated, and the strawman replays the plaintext sketch update record-by-record *inside* that update's circuit, paying its costs m times in the single proof's witness. §5 must therefore answer a question §4.1 did not address: *how are the m records that make up one update handled inside that update's circuit?* The strawman handles them sequentially, paying for m intermediate states; the rest of this section avoids that.

Our approach: certify the net effect, not the procedure. The observation that closes the gap is that the verifier does not care about $S_1, \ldots, S_{m-1}$; the verifier only cares that the published $\mathsf{Com}(S_t)$ is the result of applying $A_\tau.\mathsf{UpdateEpoch}(S_{t-1}, B_t)$. The circuit can therefore *certify the net effect* of B_t on the committed state without materialising any intermediate state: the prover supplies a compact *delta certificate* summarising how the cells of S_{t-1} map to the cells of S_t under B_t, and the circuit verifies the certificate is consistent with C_{B_t}, $\mathsf{Com}(S_{t-1})$, and $\mathsf{Com}(S_t)$. The witness shrinks from $\Theta(m|S|)$ to $\Theta(md + T)$ — the size of the certificate (md generated rows plus T touched cells) rather than the intermediate-state chain.

The rest of §5 develops this idea. §5.1 states the abstract delta-certified maintenance relation $\mathcal{R}^{\tau}_{\mathsf{upd}}$ that every sketch family's per-batch circuit must satisfy; §?? packages the per-family choices as a seven-slot interface Π_τ; §?? shows the algebra of one slot forces every supported family into one of two structural patterns; §?? and §?? walk Pattern A (Count-Min) and Pattern B (Greenwald–Khanna) end-to-end at the gate level, with §?? closing on three circuit-level optimisations; §5.4 states coverage and limits. HyperLogLog and SpaceSaving follow the same template and appear in Appendices F–G.

5.1 Batch-Effect Certificates

The transition circuit for one epoch should not replay the plaintext sketch update record by record. The verifier does not need to see the intermediate states

$$S_1, S_2, \ldots, S_{m-1}$$

inside the epoch. It only needs to be convinced that the published state commitment C_{new} is obtained by applying the declared sketch update algorithm to the old committed state C_{old} and the committed epoch C_{B_t}. We therefore use a *batch-effect certificate*: a bounded witness describing the net effect of the epoch on the sketch state.

For additive sketches such as Count-Min, the batch effect is a sparse counter-update vector: a list of sketch counters and the net amount added to each counter. For non-additive sketches, the same principle gives a different certificate shape. For example, HyperLogLog certifies a register-wise maximum update, while structured summaries such as Greenwald–Khanna certify that the new

bounded summary is a valid compressed summary of the old state plus the epoch. Thus, "batch-effect certificate" is the umbrella term; "delta" is reserved for the additive case where the effect is a sparse vector of numerical counter changes.

Abstract update relation. For a sketch family τ, the per-epoch transition circuit proves the relation

$$\mathcal{R}^{\tau}_{\mathsf{upd}} = \{(x_t, w_t) : S_t = A_\tau.\mathsf{UpdateEpoch}(S_{t-1}, B_t)\},$$

where the public input is

$$x_t = (C_{\mathsf{old}}, C_{B_t}, C_{\mathsf{new}}, \mathsf{M}),$$

and the witness w_t contains the private batch opening, the old and new sketch-state openings required by the commitment scheme, and the family-specific certificate of the batch effect. The metadata M fixes the sketch type, public parameters, hash profile, counter representation, commitment scheme, and circuit capacities.

The important point is that w_t is not a transcript of all intermediate sketch states inside the epoch. Instead, it is a compact certificate that the batch effect was computed correctly from the committed batch and applied consistently to the committed old state.

Certificate structure. The certificate is organized as a sequence of bounded witness tables and consistency checks. The input table is opened from C_{B_t}. The target table records the sketch locations affected by the epoch. The aggregate table contains one row per affected target, together with the net contribution to that target. The state-opening tables then certify that the old and new committed states differ exactly by these aggregate contributions.

For Count-Min, the target table is produced in two stages. The circuit first groups the epoch by key, obtaining rows (k_u, V_u), where V_u is the total value of key k_u in the epoch. It then computes the d counter locations $(i, h_i(k_u))$ for each grouped key and groups these rows by counter identifier. The resulting aggregate table contains one row $(i, b, \Delta_{i,b})$ for each touched counter, where

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u)=b} V_u.$$

The first grouping avoids hashing duplicate keys repeatedly; the second grouping combines hash collisions between distinct keys.

Generic certificate obligations. Although the concrete tables differ by sketch family, every batch-effect certificate must establish four facts.

(1) **Input binding.** The private batch rows used by the circuit open from the public commitment C_{B_t}. The prover cannot invent, drop, or modify records without breaking the commitment opening.
(2) **Effect derivation.** The proposed batch effect is derived from the committed rows under the declared sketch algorithm and metadata M. For Count-Min, this means that each distinct key is mapped to one counter per sketch row by the declared hash functions. For other sketches, this slot is instantiated by the corresponding register, summary, or rank-update rule.
(3) **Effect aggregation.** If several rows contribute to the same update target, the circuit verifies that they are combined correctly. For additive sketches this is a sum; for max-based sketches it is a maximum; for structured summaries it is a family-specific invariant check.
(4) **State rebinding.** The old and new state commitments are consistent with applying the certified effect. For Pattern A linear sketches, this can be a sparse additive patch. For Pattern A max sketches, this is a register-wise max patch. For Pattern B sketches, this is a whole-state replacement checked against the summary invariant.

Algorithm 1: Batch-effect update circuit, sketch-agnostic

Input: Public $C_{B_t}, C_{\mathsf{old}}, C_{\mathsf{new}}, \mathsf{M}$; private certificate tables W_τ
Output: Proof that C_{new} is obtained by applying epoch B_t to C_{old} under the declared sketch family A_τ

1 Verify that the private batch table opens from C_{B_t}, with canonical dummy rows for inactive entries;
2 Use the sketch-specific target-generation gates to derive update targets from the batch rows under metadata M;
3 Verify that all contributions to the same target are combined according to the sketch-specific aggregation operator;
4 Verify the old and new state openings required by the chosen commitment scheme;
5 Verify that applying the certified batch effect to the old committed state yields the new committed state C_{new};
6 Verify metadata compatibility and expose the transition proof as the leaf payload consumed by the history-composition layer of §4.3;

The proof follows the standard extract-then-bind structure. Knowledge soundness extracts the private certificate tables. Commitment binding pins the batch opening and the state openings. The circuit constraints then force the extracted certificate to be a correct batch effect under the declared sketch algorithm, and force the new commitment to bind to the result of applying that effect. Multi-epoch history binding is handled separately by the recursive history-composition layer of §4.3.

5.2 Count-Min Circuit Design

Assume that each uploaded record is a key–value pair (k, v), where v may be positive or negative. A Count-Min sketch stores a $d \times w$ array of counters. For one record (k, v), the plaintext update adds v to one counter in each row:

$$S[i, h_i(k)] \leftarrow S[i, h_i(k)] + v, \qquad i \in [d].$$

A point query for key q returns

$$\min_{i \in [d]} S[i, h_i(q)].$$

The special case $v = 1$ gives the usual frequency-estimation setting.

The transition circuit below certifies one committed epoch

$$B_t = \{(k_j, v_j)\}_{j=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. The circuit uses a database-style maintenance plan:

$$B \xrightarrow{\text{group by key}} G \xrightarrow{\text{compute counter ids}} U_{\mathsf{ctr}} \xrightarrow{\text{group by counter}} A \xrightarrow{\text{sparse patch}} S_t.$$

The first grouping avoids repeated hash computation for duplicate keys. The second grouping is still required because different keys may collide in the same Count-Min counter.

Count-Min Update Circuit.
Public input. Batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing d, w, the hash profile, counter representation, commitment scheme, and public capacities.
Private witness. Batch table B, key-sorted table $B^{\uparrow k}$, grouped-key table G, counter-update table U_{ctr}, counter-sorted table $U_{\text{ctr}}^{\uparrow}$, sparse patch table A, and old/new opening tables O, N.
Output. A proof that C_{new} commits to the result of applying the Count-Min epoch update to C_{old} using the committed batch C_{B_t}.
Verification target. For every Count-Min counter (i, b),

$$S_t[i,b] = S_{t-1}[i,b] + \sum_{j:\, h_i(k_j)=b} v_j.$$

5.2.1 Update Circuit.

Step 1: Bind the private batch to the public commitment. The circuit starts with a private batch table

$$B[j] = (a_j, k_j, v_j),$$

where $a_j \in \{0,1\}$ is an active flag. The circuit verifies that the active rows of B open from the public batch commitment C_{B_t}, and that inactive rows are canonical dummy rows. This prevents the prover from inventing, dropping, or modifying records while preserving the same public batch commitment.

Step 2: Bring equal keys together. The prover supplies a key-sorted table $B^{\uparrow k}$. The circuit checks

$$B^{\uparrow k} \equiv B$$

as multisets, and verifies that $B^{\uparrow k}$ is sorted by key with inactive rows placed last. After this step, all records with the same key appear in one contiguous segment.

This is a standard database operation: it is a verified ORDER BY key view of the committed batch, with a permutation check ensuring that no row has been inserted or lost.

Step 3: Aggregate values for each distinct key. The circuit scans $B^{\uparrow k}$ and aggregates each equal-key segment. The output is a grouped-key table

$$G[u] = (\alpha_u, k_u, V_u),$$

where $\alpha_u \in \{0,1\}$ is an active flag and

$$V_u = \sum_{j:\, k_j = k_u} v_j.$$

Thus G contains one active row per distinct key in the epoch.

This preaggregation is sound because Count-Min updates are additive and commutative. For any key k,

$$\mathsf{Update}(k, v_1) \circ \mathsf{Update}(k, v_2) = \mathsf{Update}(k, v_1 + v_2).$$

Therefore, replacing many records with the same key by one grouped row with the summed value preserves the final sketch state. It also reduces the number of native hash computations from md to $U_{\max} d$, where $U_{\max}$ is the public padded capacity of G.

Step 4: Compute the counters updated by each distinct key. For each active grouped row $G[u] = (1, k_u, V_u)$, the circuit computes the d Count-Min bucket locations

$$b_{u,i} = h_i(k_u), \qquad i \in [d],$$

and range-checks

$$0 \le b_{u,i} < w.$$

The grouped key k_u contributes the same aggregate value V_u to one counter in each sketch row. Equivalently, the circuit forms a counter-update table

$$U_{\text{ctr}}[u,i] = (a_{u,i}, k_u, i, b_{u,i}, V_u),$$

where $a_{u,i} = \alpha_u$. Inactive grouped-key rows produce canonical dummy counter-update rows.

If delete-aware updates are enabled, the grouped value is signed:

$$V_u = \sum_{j:\, k_j = k_u} \mathrm{op}_j v_j, \qquad \mathrm{op}_j \in \{+1, -1\}.$$

The rest of the circuit is unchanged; the counter range checks simply enforce the declared signed-counter representation.

Step 5: Bring updates to the same counter together. Different keys may hash to the same Count-Min counter. Therefore, the prover supplies a counter-sorted table $U_{\text{ctr}}^{\uparrow}$, sorted by the counter identifier (i, b). The circuit verifies

$$U_{\text{ctr}}^{\uparrow} \equiv U_{\text{ctr}}$$

as multisets, including the active flag, key, row index, bucket, and value columns. It also verifies that $U_{\text{ctr}}^{\uparrow}$ is lexicographically sorted by (i, b), with inactive rows placed last.

After this step, all contributions to the same sketch counter appear in one contiguous segment. The first grouping, in Step 3, removes repeated work from duplicate input keys. This second grouping handles hash collisions between distinct keys.

Step 6: Sum the contributions for each touched counter. The circuit performs segment aggregation over $U_{\text{ctr}}^{\uparrow}$. For each contiguous segment with the same counter identifier (i, b), it computes

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u)=b} V_u.$$

The output is a sparse patch table

$$A[r] = (\beta_r, i_r, b_r, \Delta_{i_r,b_r}),$$

where $\beta_r \in \{0,1\}$ is an active flag. Each active row of A corresponds to one touched Count-Min counter, and $\Delta_{i,b}$ is the net amount by which the epoch changes that counter.

This is the only place where we need the term "delta": it denotes the net counter change after all duplicate-key aggregation and hash-collision aggregation have already been checked.

Step 7: Apply the sparse patch to the committed state. The prover supplies old and new opening tables O and N for the touched counters listed in A. For every active row $(i, b, \Delta_{i,b}) \in A$, the circuit checks

$$N[i,b] = O[i,b] + \Delta_{i,b}.$$

It also verifies that $O[i,b]$ opens correctly against C_{old}, that $N[i,b]$ opens correctly against C_{new}, and that all counters stay within the declared integer range.

If the Count-Min state is committed by a homomorphic-additive vector commitment, the sparse patch can additionally be checked by the group relation

$$C_{\text{new}} = C_{\text{old}} + \text{Com}(\Delta),$$

where Δ is the sparse vector represented by A. This avoids opening or recomputing the entire sketch state.

Step 8: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that the metadata M fixes the declared Count-Min parameters and circuit configuration: d, w, hash profile, counter representation, commitment scheme, and public capacities. The resulting transition proof is then exposed as the leaf payload consumed by the recursive history-composition layer of §4.3. Thus, local transition soundness is handled by the Count-Min update circuit, while multi-epoch history soundness is handled by the root proof $R^{(t)}$.

Running example. For the running epoch

$$(\text{ACME}, 100),\ (\text{BETA}, 50),\ (\text{ACME}, 200),\ (\text{GAMMA}, 75),\ (\text{BETA}, 30),$$

Step 3 produces

key	grouped value
ACME	300
BETA	80
GAMMA	75

instead of hashing ACME and BETA twice. If $d = 2$, $w = 4$, and

$$h_0(\text{ACME}) = 0, \quad h_1(\text{ACME}) = 3,$$

$$h_0(\text{BETA}) = 1, \quad h_1(\text{BETA}) = 0,$$

$$h_0(\text{GAMMA}) = 2, \quad h_1(\text{GAMMA}) = 2,$$

then Step 4 forms the counter updates

key	i	b	value
ACME	0	0	300
ACME	1	3	300
BETA	0	1	80
BETA	1	0	80
GAMMA	0	2	75
GAMMA	1	2	75

and Step 6 turns them into the sparse patch

counter (i, b)	$\Delta_{i,b}$
$(0, 0)$	300
$(0, 1)$	80
$(0, 2)$	75
$(1, 0)$	80
$(1, 2)$	75
$(1, 3)$	300

assuming no collision between distinct keys in this toy hash profile.

5.2.2 Circuit Checks. The eight-step Count-Min update relation decomposes into four checks. We state them at the level of relations rather than backend-specific gates; concrete arithmetizations can realize them using sorting, permutation, lookup, product-check, or AIR-style constraints.

Key-grouping check. The prover supplies a grouped-key table G. The circuit verifies that G is exactly the aggregation of the committed batch by key:

$$V_u = \sum_{j:\, k_j = k_u} v_j.$$

This check binds the grouped rows to C_{B_t} and prevents the prover from changing the multiplicity or total value of any key.

Counter-target check. For each grouped row (k_u, V_u) and each sketch row $i \in [d]$, the circuit verifies that the claimed counter identifier is

$$(i, b_{u,i}) = (i, h_i(k_u)), \qquad 0 \le b_{u,i} < w.$$

The resulting counter-update rows carry contribution V_u.

Counter-aggregation check. The prover supplies a sparse patch table A. The circuit verifies that each patch entry is the sum of all counter-update rows mapped to the same counter:

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u) = b} V_u.$$

This check is necessary because distinct keys may collide in the same Count-Min counter.

Sparse-patch check. For each active row $(i, b, \Delta_{i,b}) \in A$, the circuit verifies that the old and new committed states differ by exactly this amount:

$$S_t[i, b] = S_{t-1}[i, b] + \Delta_{i,b}.$$

The check is performed only for touched counters, together with the commitment-opening or commitment-update checks required by the chosen state commitment scheme.

Together, these four checks imply that for every counter (i, b),

$$S_t[i, b] = S_{t-1}[i, b] + \sum_{j:\, h_i(k_j) = b} v_j,$$

which is exactly the Count-Min epoch update.

5.3 Greenwald–Khanna Circuit Design

We instantiate Greenwald–Khanna (GK) using the bounded-fusion variant adopted by S^3 [38]. This is not the original two-field GK tuple. Each active tuple has the form

$$(e, g_1, g_2, \Delta_1, \Delta_2, t),$$

where e is the representative value, g_1 and g_2 encode the rank mass immediately around e, Δ_1 and Δ_2 encode rank uncertainty from neighbouring ranges, and t is the insertion timestamp used to make fusions deterministic. The table is sorted by e, padded with dummy rows, and committed at a public capacity

$$s'(N) = \left\lceil 2s \ln\left(\frac{N}{s} + 2\right) + 2 \right\rceil,$$

where N is the cumulative number of inserted records. The parameter s controls the rank-error tolerance; the running capacity $s'(N)$ grows only mildly with N/s.

The transition circuit certifies the bounded-GK batch update used in the MPC construction: insert the new epoch values into the old sorted tuple table, initialize their rank-uncertainty fields from neighbouring old tuples, perform a fixed number of adjacent-tuple fusion rounds, compact the result, and recommit the bounded

table. We use this as the declared GK update algorithm. We do not rely on GK being a commutative mergeable monoid over arbitrary summaries; for update proofs, it is enough that the declared batch-fusion procedure is deterministic and has a public capacity bound.

Greenwald–Khanna Update Circuit.
Public input. Batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, old population N_{t-1}, new population $N_t = N_{t-1} + m$, and metadata M fixing s, the capacity function $s'(N)$, value representation, commitment scheme, timestamp convention, and public table capacities.
Private witness. Batch table B, old GK table S_{old}, sorted union table U, initialized union table $U^{(0)}$, fusion-round tables $U^{(1)}, \ldots, U^{(L)}$, compacted new table S_{new}, and commitment-opening witnesses.
Output. A proof that C_{new} commits to the result of applying the declared bounded-GK epoch update to C_{old} using the committed batch C_{B_t}.
Verification target. The new committed state is the bounded-GK summary obtained by inserting all values in B_t into the old summary, initializing the new tuple fields according to the declared bounded-GK rule, applying the fixed fusion schedule, and compacting to public capacity $s'(N_t)$.

5.3.1 *Update Circuit.*

Step 1: Bind the old state and the private batch. The circuit starts with a private batch table

$$B[j] = (a_j, e_j),$$

where $a_j \in \{0, 1\}$ is an active flag and e_j is the uploaded value. It verifies that active rows of B open from the public batch commitment C_{B_t}, and that inactive rows are canonical dummies. The circuit also verifies that the old tuple table

$$S_{\text{old}}[r] = (a_r, e_r, g_{1,r}, g_{2,r}, \Delta_{1,r}, \Delta_{2,r}, t_r)$$

opens from C_{old}, is sorted by (e, t), and is padded to the public old capacity $s'(N_{t-1})$. These checks bind the transition to the committed old state and the committed epoch.

Step 2: Insert the epoch values into the old tuple order. The prover supplies a sorted union table U of capacity

$$M = s'(N_{t-1}) + m.$$

The active rows of U consist of all active old tuples and one fresh tuple for each active batch value. A fresh tuple for $B[j] = (1, e_j)$ has timestamp $t = N_{t-1} + j$ and placeholder rank fields:

$$(e_j, \bot, \bot, \bot, \bot, N_{t-1} + j).$$

The circuit verifies two facts: first, U is a multiset union of the old tuples and the fresh batch tuples; second, U is sorted by (e, t), with inactive rows placed last. Timestamps make the ordering deterministic even when several equal values appear.

Step 3: Initialize the new tuple fields. For every fresh tuple, the bounded-GK rule sets

$$g_1 = 0, \qquad g_2 = 0.$$

The uncertainty fields are copied from the nearest old tuple on each side. Let $R^+(r)$ be the nearest active old tuple after row r in U, and let $R^-(r)$ be the nearest active old tuple before row r. For a fresh tuple at row r, the circuit checks

$$\Delta_1[r] = \begin{cases} R^+(r).g_1 + R^+(r).\Delta_1, & \text{if } R^+(r) \text{ exists,} \\ 0, & \text{otherwise,} \end{cases}$$

and

$$\Delta_2[r] = \begin{cases} R^-(r).g_2 + R^-(r).\Delta_2, & \text{if } R^-(r) \text{ exists,} \\ 0, & \text{otherwise.} \end{cases}$$

Old tuples keep their original fields. The initialized table is denoted $U^{(0)}$.

This step can be checked by prefix-copy and suffix-copy certificates: one pass propagates the nearest preceding old tuple information, and one pass propagates the nearest following old tuple information. The public shape of the check depends only on M, not on the private locations of the inserted values.

Step 4: Fix the fusion threshold and number of rounds. After the epoch is inserted, the new population is

$$N_t = N_{t-1} + m.$$

The fusion threshold is

$$\theta_t = \left\lfloor \frac{N_t}{s} \right\rfloor.$$

The circuit uses a public number of fusion rounds

$$L = \lceil \log M \rceil, \qquad M = s'(N_{t-1}) + m.$$

This is a conservative public bound on the number of parallel adjacent fusion rounds needed to complete all eligible fusions in the union table. The S^3 analysis uses the same bounded-fusion principle: repeated parallel fusions reduce any fully fusible run geometrically, so a logarithmic number of rounds suffices.

Step 5: Certify adjacent tuple fusions. The prover supplies tables

$$U^{(1)}, U^{(2)}, \ldots, U^{(L)}$$

representing the table after each fusion round. Each round applies a fixed adjacent-pair schedule, alternating parity so that every adjacent pair is checked across the two passes of a round.

For two adjacent active tuples

$$R_1 = (e_1, g_{1,1}, g_{2,1}, \Delta_{1,1}, \Delta_{2,1}, t_1)$$

and

$$R_2 = (e_2, g_{1,2}, g_{2,2}, \Delta_{1,2}, \Delta_{2,2}, t_2),$$

the circuit verifies exactly one of the following outcomes.

If $t_1 > t_2$, then the newer left tuple may be fused into the right tuple when

$$g_{1,1} + g_{2,1} + g_{1,2} + \Delta_{1,2} + 1 \le \theta_t.$$

In that case, R_1 becomes dummy and R_2 is updated by

$$g'_{1,2} = g_{1,1} + g_{2,1} + g_{1,2} + 1,$$

while the representative value, timestamp, and uncertainty fields of R_2 remain those of R_2.

Symmetrically, if $t_2 > t_1$, then the newer right tuple may be fused into the left tuple when

$$g_{1,2} + g_{2,2} + g_{2,1} + \Delta_{2,1} + 1 \le \theta_t.$$

In that case, R_2 becomes dummy and R_1 is updated by

$$g'_{2,1} = g_{1,2} + g_{2,2} + g_{2,1} + 1.$$

If the relevant inequality fails, or if one of the two rows is inactive, both rows are copied forward unchanged. These checks certify the same local FuseTuple rule as the declared bounded-GK update.

Step 6: Compact and recommit the bounded state. After the L fusion rounds, the prover supplies a compacted table S_{new}. The circuit verifies that S_{new} is the stable compaction of the active rows of $U^{(L)}$, padded to the public new capacity

$$s'(N_t) = \left\lceil 2s \ln\left(\frac{N_t}{s} + 2\right) + 2 \right\rceil.$$

The circuit then verifies that S_{new} opens to the published commitment C_{new}. This is a whole-state replacement check rather than a sparse additive patch: GK is an ordered summary whose correctness is global, not a fixed array of independent counters.

Step 7: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that the metadata M fixes the declared GK variant, including s, the capacity function $s'(N)$, tuple layout, timestamp convention, fusion threshold, fusion schedule, commitment scheme, and public table capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Running example. We illustrate the update circuit with the same toy shape as the bounded-GK merge protocol of S^3 [38]. Suppose the old committed GK state contains four active tuples:

e	g_1	g_2	Δ_1	Δ_2	t
3	1	1	0	1	5
7	2	3	1	0	10
12	1	1	2	3	15
18	1	0	0	2	2

and the new epoch contains

$$B_t = (1, 5, 15, 6).$$

Assume the old population is $N_{t-1} = 16$, the new timestamps are 17, 18, 19, 20, and the fusion threshold for this toy example is

$$\theta_t = \frac{N_t}{s} = 2.$$

This small instance is for illustration only; in the actual circuit the state table is padded to the public capacity $s'(N_t)$.

First, the circuit verifies that the sorted union table contains all old tuples and one fresh tuple for each epoch value. Fresh tuples have placeholder rank fields:

e	g_1	g_2	Δ_1	Δ_2	t
1	$\perp$	$\perp$	$\perp$	$\perp$	17
3	1	1	0	1	5
5	$\perp$	$\perp$	$\perp$	$\perp$	18
6	$\perp$	$\perp$	$\perp$	$\perp$	20
7	2	3	1	0	10
12	1	1	2	3	15
15	$\perp$	$\perp$	$\perp$	$\perp$	19
18	1	0	0	2	2

The circuit checks that this table is a multiset union of the old state and the committed epoch values, and that it is sorted by (e, t).

Next, the circuit initializes the fresh tuples. Each fresh tuple gets $g_1 = g_2 = 0$. Its Δ_1 value is copied from the nearest old tuple on the right as $g_1 + \Delta_1$, and its Δ_2 value is copied from the nearest old tuple on the left as $g_2 + \Delta_2$. The initialized table is:

e	g_1	g_2	Δ_1	Δ_2	t
1	0	0	1	0	17
3	1	1	0	1	5
5	0	0	3	2	18
6	0	0	3	2	20
7	2	3	1	0	10
12	1	1	2	3	15
15	0	0	1	4	19
18	1	0	0	2	2

For example, the fresh tuple $e = 5$ lies between old tuples 3 and 7. Hence

$$\Delta_1(5) = g_1(7) + \Delta_1(7) = 2 + 1 = 3,$$

and

$$\Delta_2(5) = g_2(3) + \Delta_2(3) = 1 + 1 = 2.$$

The fresh tuple $e = 6$ has the same nearest old neighbours, so it gets the same uncertainty values. The fresh tuple $e = 15$ lies between old tuples 12 and 18, giving

$$\Delta_1(15) = g_1(18) + \Delta_1(18) = 1 + 0 = 1,$$

and

$$\Delta_2(15) = g_2(12) + \Delta_2(12) = 1 + 3 = 4.$$

The circuit then verifies the fixed fusion schedule. In this example, two fresh tuples can be fused into neighbouring old tuples under the threshold $\theta_t = 2$. The fresh tuple $e = 1$ is newer than the tuple $e = 3$, and

$$0 + 0 + 1 + 0 + 1 = 2 \le \theta_t.$$

Therefore $e = 1$ is fused into $e = 3$, making the first row dummy and updating

$$g_1(3) \leftarrow 0 + 0 + 1 + 1 = 2.$$

Similarly, the fresh tuple $e = 15$ is newer than $e = 18$, and

$$0 + 0 + 1 + 0 + 1 = 2 \le \theta_t.$$

Thus $e = 15$ is fused into $e = 18$, making the $e = 15$ row dummy and updating

$$g_1(18) \leftarrow 0 + 0 + 1 + 1 = 2.$$

The post-fusion table is:

e	g_1	g_2	Δ_1	Δ_2	t
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
3	2	1	0	1	5
5	0	0	3	2	18
6	0	0	3	2	20
7	2	3	1	0	10
12	1	1	2	3	15
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
18	2	0	0	2	2

Finally, the circuit verifies stable compaction: dummy rows are removed, active rows keep their relative order, and the result is

padded to the public capacity. The new committed GK state is:

e	g_1	g_2	Δ_1	Δ_2	t
3	2	1	0	1	5
5	0	0	3	2	18
6	0	0	3	2	20
7	2	3	1	0	10
12	1	1	2	3	15
18	2	0	0	2	2

The proof therefore certifies the whole batch effect: the epoch values were inserted into the old ordered summary, their uncertainty fields were initialized from the correct neighbouring old tuples, all eligible FuseTuple steps in the public schedule were applied correctly, and the compacted table is exactly the new committed GK state.

Summary. The GK transition circuit proves five logical claims. First, the old state and epoch rows are bound to their public commitments. Second, the union table is the sorted insertion of the epoch values into the old tuple table. Third, the rank-uncertainty fields of fresh tuples are initialized from neighbouring old tuples according to the bounded-GK rule. Fourth, the fixed fusion schedule correctly applies the local FuseTuple rule under threshold $\theta_t = \lfloor N_t/s \rfloor$. Fifth, the compacted bounded table is exactly the new committed state. Together these claims certify that C_{new} is the result of the declared bounded-GK batch update.

5.3.2 Circuit Checks. The update relation above can be summarized by four checks. We state them at the relation level, independent of any particular proof backend.

Insertion-order check. The circuit verifies that the sorted union table contains exactly the old tuples and the new batch tuples, and that it is ordered by (e, t). This replaces the data-dependent insertion positions of the plaintext algorithm with a single ordered witness table.

Initialization check. The circuit verifies that each new tuple has $g_1 = g_2 = 0$, and that Δ_1, Δ_2 are copied from the nearest old tuple on the right and left, respectively. This is the certificate analogue of the suffix-copy and prefix-copy initialization used in the bounded-GK streaming protocol.

Fusion check. The circuit verifies a public number of adjacent fusion rounds. Each pair either remains unchanged or is fused according to the timestamp order and the threshold inequalities above. Since the number of rounds is public, the circuit shape does not reveal how many fusions actually occurred.

Replacement check. The circuit verifies that the new committed state is the stable compaction of the final fusion table, padded to $s'(N_t)$. Unlike Count-Min, there is no per-cell delta vector; the certificate proves a valid replacement of the whole bounded summary.

5.3.3 Rank Query Circuit. A GK rank query asks for the approximate rank of a query value q. In zkSketch, queries are answered against a closed sketch version: if an epoch is open when the query arrives, it is first closed and proved. Therefore, the query circuit reads only the committed GK state. If a deployment keeps an explicit unmerged buffer, the buffer contribution can be added by the same comparison-and-sum check used in S^3.

The prover opens the committed GK table S_t and supplies selector bits identifying the adjacent tuple pair that brackets q. Let i be the selected index such that

$$e_i \le q < e_{i+1}.$$

The circuit verifies the one-hot selector and the two comparisons above. It also verifies the prefix sums

$$P_i = \sum_{j \le i} \big(g_{1,j} + g_{2,j} + 1\big).$$

Then it computes

$$r_{\min} = P_i - \Delta_{2,i} - g_{2,i},$$

and

$$r_{\max} = P_i + \Delta_{1,i+1} + g_{1,i+1}.$$

The public answer is

$$ans = \frac{r_{\min} + r_{\max}}{2},$$

or the deterministic rounded version specified by the public release policy. The proof therefore certifies that the answer is the declared bounded-GK rank estimate for the committed state version.

Greenwald–Khanna Rank Query Circuit.
Public input. State commitment C_{state}, query value q, metadata M, and public answer ans.
Private witness. Opened GK table S_t, opening witnesses, prefix-sum witness P, selector bits for the bracketing pair, and intermediate values $r_{\min}, r_{\max}$.
Verification target. The opened table is the committed GK state, the selected adjacent tuple pair brackets q, and

$$ans = \frac{(P_i - \Delta_{2,i} - g_{2,i}) + (P_i + \Delta_{1,i+1} + g_{1,i+1})}{2}.$$

Checks. The circuit checks the state opening, tuple ordering, one-hot bracket selection, prefix-sum recurrence, the formulas for $r_{\min}$ and $r_{\max}$, and the final answer computation.

5.3.4 Remark on Mergeability. The bounded-GK variant above is suitable for our update circuit because it gives a deterministic, capacity-bounded batch-fusion procedure. In this subsection, "merge" means merging a batch of new values into one existing GK summary, as in the S^3 protocol. We do not require the stronger property that two arbitrary independently maintained GK summaries form a commutative mergeable monoid. If a deployment needs cross-shard GK merging, it should either use a separately declared GK summary-merge algorithm and prove that relation, or close each shard to a committed epoch sequence and combine histories at the framework layer. The transition circuit here only assumes the bounded batch-update procedure specified above.

5.4 Coverage and Other Sketches

The remaining two families fill the same seven slots. HyperLogLog (Pattern A max with a leading-run hash) is in Appendix F; Space-Saving (Pattern B with top-s retention via the threshold-residual certificate, in place of GK's rank-interval fusion) is in Appendix G. The framework supports any sketch whose update is expressible as either Pattern A or Pattern B with a constant-depth aggregation gate; counting Bloom variants, MinHash, KLL [18], t-digest, and the broader class of mergeable summaries [1] are direct extensions,

with KLL worked through as the extensibility example in §**??**. Out of scope: sketches whose merge invariant is non-constant-depth, online learned summaries with adaptive structure, and sketches whose accuracy guarantee relies on randomness that cannot be committed in advance under HPP.

6 Evaluation (in progress)

This section describes the evaluation plan. Empirical results are not included in this draft; they will appear in the camera-ready version. The analytical evaluation in Appendix B grounds the design choices in constraint-count derivations validated against published Halo2 cost microbenchmarks, and Table **??** reports per-sketch planner outputs under three deployment profiles. A convincing SIGMOD/VLDB evaluation should validate the comparative analysis empirically and quantify each compositional and framework-level mechanism's contribution; we organise the planned experiments around three research questions.

6.1 Research Questions

The evaluation answers three research questions that mirror the contribution structure of §1.

- **RQ1 (Capability).** Does delta-certified maintenance preserve sketch accuracy and produce checkable proofs across all four families (CM, HLL, SS, GK), under representative workloads drawn from the regulated-reporting deployment of Example **??**?
- **RQ2 (Scalability).** How do prover time, verifier time, proof size, and memory scale along each cost dimension — state size s, depth d, batch size m, batch rate b, audit-window length E, coupled-sketch count K — compared to optimised exact-ZK-index and zkVM baselines?
- **RQ3 (Component contribution).** How much does each compositional technique (SDC, HPP, NDC, CCS) and each framework-level mechanism (deployment-aware compiler, workflow decomposition) contribute to the end-to-end cost, measured against the optimised single-sketch baseline and against the `naive-zkSketch` strawman of §**??** as a rhetorical upper bound?

6.2 Experimental Setup

Implementation. The prototype targets a Halo2-style PLONKish backend in Rust, reusing standard gates for range checks, sorting, lookups, equality flags, and arithmetic aggregation, following the methodology of PoneglyphDB and VPJoin [2, 17]. The implementation exposes the slot catalog of Appendix D so the deployment-aware compiler of §**??** can pick layouts per deployment profile.

Implementation roadmap. The implementation is staged in three increments. *Stage 1 (per-sketch update circuits):* implement CM, HLL, SS, GK update gates against a single-sketch baseline; validate against the comparative analysis (RQ1 and the small-s regime of RQ2). *Stage 2 (compositional techniques):* add SDC, HPP, NDC, CCS as planner-controlled extensions; run the per-technique ablation of RQ3. *Stage 3 (planner and workflows):* add the deployment-aware compiler, the three-dimensional cost surface, and workflow decomposition; run the full RQ2 sweep and the RQ3 framework-mechanism ablation. The Halo2 backend choice is the simplest path to a working prototype because PoneglyphDB and ZKSQL provide reference implementations of the relational operator gates we reuse; we do not commit to a specific Halo2 fork (PSE, Axiom, or Scroll).

Reproducibility plan. The implementation, benchmark scripts, datasets (synthetic generators + pointers to public real datasets), and per-experiment configuration files will be released under an open-source license at the time of submission. Evaluation tables will report median and 95%-quantile of five trials; we will include a per-experiment cost-model prediction column to validate the analytical model of §**??** against measured numbers.

Workloads. (i) Frequency / range-frequency over filings/transactions (CM); (ii) distinct counts over high-cardinality keys (HLL); (iii) top-k heavy hitters (SS); (iv) quantile queries on numerical fields (GK); (v) multi-sketch group-by exercising SDC; (vi) insert-heavy and delete-aware mixes exercising NDC; (vii) release policies (thresholding, rounding, optional noisy release).

Datasets. Synthetic uniform, Zipfian, and Gaussian record streams to control skew and collision behaviour; real datasets matching the deployments (SEC filings or simulated regulatory data; on-chain transactions; network flow traces). Dataset choice exercises both high-cardinality and skewed regimes.

Baselines. (i) Naive exact ZK index (Merkle leaves); (ii) optimised exact ZK index (sorted-index commitment with lookups, in the style of PoneglyphDB and ZKSQL); (iii) optimised sketch (this paper, with all four compositional techniques and the planner enabled); (iv) zkVM baselines (RISC0 and SP1) [31, 34]; (v) plaintext sketch engine (Apache DataSketches [3]) as a lower-bound reference. The `naive-zkSketch` strawman of §**??** appears as a rhetorical reference point in the RQ3 discussion; its asymptotic cost is derived analytically in Appendix B rather than measured, because its $\Theta(mdH \log w)$ per-batch cost would dominate any practical sweep before reaching the regime where the optimised sketch operates. Reporting baselines that run out of memory or time as `OOM/TLE` rather than silently skipping them, following the convention of ZK-Value [36], makes the comparison's failure modes explicit.

Metrics. Circuit rows, custom-gate counts, lookup counts, prover time, witness-generation time, verification time, proof size, memory, state size, commitment-opening cost, query latency, batch update throughput, touched-cell count, approximation error. Per technique: SDC sharing factor as K varies; HPP multi-opening cost vs. batch size; NDC overhead vs. deletion fraction; CCS DP budget vs. padding overhead. For the planner: the three-coordinate cost surface (§**??**) instantiated for the three deployment profiles.

6.3 RQ1 — Capability

RQ1 is answered by running each of the four families against workload classes (i)–(iv). For each family the experiment certifies four things: (a) the update circuit accepts on honest witnesses across the full sweep of s, d; (b) the decode circuit reproduces the plaintext approximate answer to within the family's stated error bound; (c) the sketch-merge segment tree of §4.3 commits an audit window of E batches with verifier-side cost matching the predicted $O(1)$ (root proof) or $O(\log E)$ (per-batch maintenance); (d) a tampered batch — 5% row substitution, mirroring the fraud scenario of Example **??** — is rejected by the verifier with overwhelming probability.

Takeaway for RQ1. Expected: zkSketch produces checkable proofs for every workload across the four families, and tampering with 5% of rows in any batch is detected via the commitment-binding chain (requirement R-2 of §1). The approximation error matches the family's analytical bound (a sketch property, not a ZK one).

6.4 RQ2 — Scalability

RQ2 is answered by sweeping each cost dimension while holding the others fixed at the running-example default, mirroring the dimension-by-dimension panels used by zk-Value (Figure 3 of [36]). Specifically: $s \in \{2^{10}, \ldots, 2^{18}\}$ for state size; $d \in \{1, \ldots, 16\}$ for depth; $m \in \{2^6, \ldots, 2^{14}\}$ for batch size; $b \in \{1, \ldots, 10^3\}$ batches per hour for batch rate; $E \in \{1, \ldots, 10^4\}$ for audit-window length; $K \in \{1, 2, 3, 5\}$ for coupled sketches. For each sweep we report prover time, verifier time, proof size, and peak memory; the optimised exact-ZK-index and zkVM baselines provide the comparison.

Takeaway for RQ2. Expected: prover time scales linearly in m and T and sub-linearly in s (touched-cell-only commitment work); verifier time scales linearly in E, or logarithmically with the segment-tree extension; proof size is constant per batch (single Halo2 proof). The per-record update cost ratio against the optimised exact index grows with n in the optimised sketch's favour, validating requirement R-3 of §1.

6.5 RQ3 — Component contribution

RQ3 is answered by ablating each component on the workload it targets, with the optimised single-sketch baseline as the reference and the `naive-zkSketch` strawman of §?? as the rhetorical upper bound. *SDC*: vary $K \in \{1, 2, 3, 5\}$ on multi-sketch group-by, measuring the sort-share reduction predicted in §??. *HPP*: vary m and d on Count-Min with hash-profile sizes $N \in \{2^{16}, 2^{20}\}$, measuring the crossover where committed-lookup pays back its $\Theta(N \cdot H)$ preprocessing. *NDC*: vary deletion fraction $f \in \{0, 0.1, 0.5\}$, validating that signed deltas match additive cost up to the range-check overhead. *CCS*: vary DP scale σ across the static / adaptive / DP-via-CCS policies of Table 8, validating the predicted $\sim 1\%$ auxiliary-sketch overhead and the $(\varepsilon_N, \delta_N)$ leakage accounting. *Compiler*: contrast planner-chosen layouts across the three deployment profiles against a fixed-baseline layout. *Workflow decomposition*: vary E and shard count K to validate the flat-DAG composition cost.

We additionally ablate the three Count-Min circuit-level optimisations of §??; the same knobs reappear in the appendix instantiations for HLL and SS (Appendices F and G) and reuse the same experimental machinery. *Wide-output Poseidon2* (§??): sweep depth $d \in \{2, 4, 7, 16\}$ on the CM update circuit under NativeHash; measure Poseidon2 permutation count per batch with and without the wide-output trick. Predicted reduction from md to m permutations per batch ($d\times$ on the dominant cost line under NativeHash). Re-run on HLL with $d = 2$ outputs per record for the $2\times$ predicted reduction. *LogUp routing* (§??): compare the baseline sort+segment pipeline against LogUp routing on the CM update circuit at $m \in \{1024, 16384\}$, $d = 7$; report constraint count, column-family count, and prover wall time. Predicted near-equivalent total cost with ~30% fewer column families. Re-run on the SS-Group step. *Batch deduplication* (§??): sweep Zipf parameter $\alpha \in \{0, 0.5, 1, 1.5\}$ on $m = 16{,}384$, $d = 7$; measure realised distinct-key count u and Poseidon2 invocations with wide-output + dedup vs wide-output alone. Predicted $4\times$–$8\times$ hash-cost reduction at $\alpha = 1$ on the AOL clickstream benchmark of [38]. Re-run on HLL and SS with the family-appropriate cost line.

Takeaway for RQ3. Expected: each compositional technique delivers its predicted asymptotic saving on its target workload; each circuit-level optimisation delivers its predicted hash-count or column-count reduction at its activation conditions; the planner chooses materially different layouts across deployment profiles with measurable per-profile cost gaps; and the gap between the optimised sketch and `naive-zkSketch` approaches the analytical $\sim 80\times$ derived in §??. Together these confirm that each framework component pays for itself in the regime where it activates.

6.6 RQ4 — Composition topology trade-off

RQ4 validates the design-space summary of §4.3 by measuring per-update cost and query latency under the three composition topologies (linear chain, balanced binary tree, streaming forest) and the two peak-chain policies (lazy vs. eager). The sweeps fix CM as the per-batch update circuit — the topology choice is sketch-agnostic — and vary the topology and policy as orthogonal axes.

4-a. Insert-rate scaling on a single-core prover. Sweep batch rate $b \in \{1, 10, 10^2, 10^3\}$ batches/sec on one core; report per-update online cost for the chain (one merge), the balanced binary tree (the $\Theta(\log E)$ spine refresh at $E = 1024$), and the streaming forest (cascade amortised and worst-case). Expected: the chain wins at low rate, the streaming forest's amortised $O(1)$ wins at high rate; the binary tree sits in between under its known-E caveat.

4-b. Insert-rate scaling on a multi-core prover. Repeat 4-a on a 16-core machine. Expected: the chain stays single-threaded and saturates at the same rate as 4-a; the balanced binary tree and the streaming forest scale to parallelism, validating the parallelisation motivation of §4.3.

4-c. Query latency under lazy vs. eager peak chain. Sweep query rate q from 1/min to 1/sec while holding the insert rate at 10^2 batches/sec; measure query latency (from query arrival to verifier-checked root proof) and P99 insert latency (cascade-aligned bursts). Expected: under lazy, query latency follows $O(\log t)$ and P99 insert latency follows the cascade depth; under eager, query latency is $O(1)$ and P99 insert latency picks up the cascade-rebuild surcharge. The crossover at which eager dominates matches the predicted query-to-insert ratio threshold of §4.3.

Takeaway for RQ4. Expected: the topology and peak-chain policy each have a measurable crossover point on the batch-rate / query-rate plane, and the policy default proposed in §4.3 (streaming forest with eager peak chain) sits on the Pareto frontier for streaming SLAs with unpredictable query arrival.

Table ?? maps each RQ-level claim to the experiment that validates it.

7 Related Work

MPC sketches and streaming ZK. S^3 [38] designs efficient sketch protocols in outsourced MPC and is the closest algorithmic precedent: its batch-update insight (sort, segmented prefix-sum/max,

compact, top-s retain, GK fusion rounds) reappears throughout our certificate-checking pipeline. zkSketch differs on four dimensions: *trust model* (S^3 is multi-server semi-honest MPC; zkSketch is single-prover non-interactive ZK against malicious provers), *output shape* (secret shares vs. transferable proofs), *state* (fresh per-batch vs. committed and tree-bound), and *cost model* (MPC rounds and bandwidth vs. PLONKish constraints and proof size). The algorithmic primitives transfer; the cryptographic regime is fundamentally different. Whisper [30] combines streaming, sketching, and silently verifiable proofs with multiple non-colluding servers; streaming ZK proofs provide foundations for low-space verifiers [7]; both target high-throughput streaming with multi-party trust and are not direct comparators for the dynamic single-prover regime.

Verification-over-computation as a technique. Mystique [37], zkCNN [23], and vCNN [21] apply VoC pointwise (Freivalds' algorithm [11]) to one or two operators each. zkSketch elevates the technique to a framework discipline organized by certificate shape (§**??**).

Sketches and AQP. Count-Min, HyperLogLog, SpaceSaving, and GK are classic compact summaries [8, 10, 15, 25]; KLL [18] and the broader mergeable-summaries class [1] are natural extensions; Apache DataSketches packages many in production form [3]. zkSketch differs by adding committed sketch state, a maintained sketch-merge segment tree for verifiable history binding, and verifiable updates over a single untrusted prover.

Recursion, folding, and proof pipelines. Recursive proofs, folding, IVC, and accumulation [6, 19, 20, 32] carry validity across long computations. zkSketch treats them as first-class composition primitives: the three composition topologies of §4.3 place chained / folding, binary-tree, and k-ary aggregation in one design space and characterise the trade-offs against batch size, parallelism, and core count. Reckle Trees [28] apply a single Merkle-tree topology to a static rollup; modern zkVMs and proof-pipeline aggregators [27, 29, 31, 33, 34] adopt k-ary recursion for general-purpose computation. zkSketch is, to our knowledge, the first work to characterise these topologies for the specific workload of streaming sketch maintenance under non-interactive ZK.

References

[1] Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. 2012. Mergeable Summaries. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 23–34. https://doi.org/10.1145/2213556.2213562

[2] Anonymous Authors. 2026. VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs. Manuscript supplied by the authors.

[3] Apache Software Foundation. 2026. Apache DataSketches. https://datasketches.apache.org/ Accessed 2026-05-03.

[4] Jon Louis Bentley and James B. Saxe. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.

[5] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. 2017. The hunting of the SNARK. *Journal of Cryptology* 30, 4 (2017), 989–1066.

[6] Benedikt Bünz and Binyi Chen. 2023. ProtoStar: Generic Efficient Accumulation/Folding for Special-Sound Protocols. In *Advances in Cryptology – ASIACRYPT 2023*. 77–110. https://doi.org/10.1007/978-981-99-8724-5_3

[7] Graham Cormode, Marcel Dall'Agnol, Tom Gur, and Chris Hickey. 2024. Streaming Zero-Knowledge Proofs. In *39th Computational Complexity Conference (LIPIcs)*, Vol. 300. 2:1–2:66. https://doi.org/10.4230/LIPIcs.CCC.2024.2

[8] Graham Cormode and S. Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

[9] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.

[10] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Analysis of Algorithms 2007*. 127–146. https://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf

[11] Rūsiņš Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. *Information Processing 77, Proceedings of IFIP Congress 77* (1977), 839–842.

[12] Ariel Gabizon and Zachary J. Williamson. 2020. Plookup: A Simplified Polynomial Protocol for Lookup Tables. IACR Cryptology ePrint Archive, Report 2020/315. https://eprint.iacr.org/2020/315

[13] S. Goldwasser, S. Micali, and C. Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA*.

[14] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. 2019. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*. 203–225.

[15] Michael Greenwald and Sanjeev Khanna. 2001. Space-Efficient Online Computation of Quantile Summaries. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. 58–66. https://doi.org/10.1145/375663.375670

[16] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 305–326.

[17] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-Interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1, Article 63 (2025). https://doi.org/10.1145/3709713

[18] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal Quantile Approximation in Streams. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. 71–78. https://doi.org/10.1109/FOCS.2016.17

[19] Abhiram Kothapalli and Srinath Setty. 2022. SuperNova: Proving Universal Machine Executions without Universal Circuits. IACR Cryptology ePrint Archive, Report 2022/1758. https://eprint.iacr.org/2022/1758

[20] Abhiram Kothapalli and Srinath Setty. 2023. HyperNova: Recursive Arguments for Customizable Constraint Systems. IACR Cryptology ePrint Archive, Report 2023/573. https://eprint.iacr.org/2023/573

[21] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760

[22] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816. https://doi.org/10.14778/3594512.3594513

[23] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2968–2985. https://doi.org/10.1145/3460120.3485379

[24] Claire Mathieu, Rajmohan Rajaraman, Neal E. Young, and Arman Yousefi. 2024. Competitive Data-Structure Dynamization. *ACM Transactions on Algorithms* 20, 4 (2024).

[25] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2006. An Integrated Efficient Solution for Computing Frequent and Top-k Elements in Data Streams. *ACM Transactions on Database Systems* 31, 3 (2006), 1095–1133. https://doi.org/10.1145/1166074.1166084

[26] Mina Foundation. 2024. Scan State: Parallel Recursive Proof Composition for the Mina Protocol. https://docs.minaprotocol.com/mina-protocol/scan-state.

[27] OpenVM. 2026. Generating Proofs. https://docs.openvm.dev/book/writing-apps/generating-proofs/ Accessed 2026-05-03.

[28] Charalampos Papamanthou, Shravan Srinivasan, Nicolas Gailly, Ismael Hishon-Rezaizadeh, Andrus Salumets, and Stjepan Golemac. 2024. Reckle Trees: Updatable Merkle Batch Proofs with Applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM. https://doi.org/10.1145/3658634.3670354

[29] Polygon Labs. 2026. Polygon zkEVM Proving Architecture. https://docs.polygon.technology/tools/zkevm/architecture/zkprover/stark-recursion/proving-architecture/ Accessed 2026-05-03.

[30] Mayank Rathee, Yuwen Zhang, Henry Corrigan-Gibbs, and Raluca Ada Popa. 2024. Private Analytics via Streaming, Sketching, and Silently Verifiable Proofs. IACR Cryptology ePrint Archive, Report 2024/666. https://eprint.iacr.org/2024/666

[31] RISC Zero. 2026. Recursive Proving Documentation. https://dev.risczero.com/api/recursion Accessed 2026-05-03.

[32] Srinath Setty. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Advances in Cryptology – CRYPTO 2022*. 359–388. https://doi.org/10.1007/978-3-031-15985-5_13

[33] Starknet. 2026. SHARP: Starknet Proof Aggregator Documentation. https://docs.starknet.io/learn/protocol/sharp Accessed 2026-05-03.

[34] Succinct. 2026. SP1 Proof Aggregation Documentation. https://docs.succinct.xyz/docs/sp1/writing-programs/proof-aggregation Accessed 2026-05-03.
[35] Peter Todd. 2016. Merkle Mountain Ranges. OpenTimestamps technical specification. https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md
[36] Zhaoyu Wang, Pingchuan Ma, Zhantong Xue, Yuguang Zhou, Qixin Zhang, Xiaoqin Zhang, and Shuai Wang. 2026. ZK-Value: A Practical Zero-Knowledge System for Verifiable Data Valuation. arXiv preprint arXiv:2605.03581. https://arxiv.org/abs/2605.03581
[37] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. 501–518.
[38] Jianzhe Yu, Feng Han, Qi Dong, Qiyao Luo, Weiran Liu, Lin Qu, and Ke Yi. 2026. Sketch-Based Secure Query Processing for Streaming Data. *Proceedings of the ACM on Management of Data* 4, 3, Article 233 (2026). https://doi.org/10.1145/3802110
[39] Zcash Foundation. 2026. The halo2 Book. https://zcash.github.io/halo2/ Accessed 2026-05-03.

A Framework Guarantees

The framework's guarantees follow by composing local transition soundness with recursive history soundness.

Local transition soundness. For every update j, the per-update proof π_j is knowledge-sound for $\mathcal{R}^{\tau}_{\mathsf{upd}}$. Therefore, an accepting proof binds $\mathsf{Com}(S_j)$ to $\mathsf{Com}(S_{j-1})$ and C_{b_j} under the declared update algorithm. A prover who fabricates the new state, omits a record from the committed update, or applies the wrong sketch algorithm cannot produce an accepting proof except with negligible probability.

History soundness. For every interval $[i, j]$, the recursive proof $\Pi_{[i,j]}$ verifies the child interval proofs and enforces equality of adjacent boundary state commitments. By induction on the interval tree, the root proof $\Pi_{[1,t]}$ certifies that $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying the committed updates $b_1, \ldots, b_t$ in order.

Query-prefix soundness. For a query at logical time t, the prover first closes all uploads before t into committed updates and proves the resulting history root. The decode proof then certifies the answer against the corresponding state commitment. Hence an accepting query proof is tied to exactly the uploaded prefix visible to that query.

Tamper resistance. Dropping an update, reordering two updates, replacing an update commitment, or substituting an intermediate state changes either a leaf public input, an interval boundary, or the ordered digest $H_{[1,t]}$. Under commitment binding and hash collision resistance, such a change cannot produce the same root commitment $R^{(t)}$ with an accepting root proof, except with negligible probability.

Succinct verification. The verifier checks only the root proof $\Pi^{(t)}$ against the root commitment $R^{(t)}$, the current state commitment $\mathsf{Com}(S_t)$, and the public metadata. Verification cost is independent of the number of updates in the verification window and independent of the number of records inside those updates.

Privacy. Zero knowledge hides the update witnesses, including raw records, sketch states, delta tables, sorted certificates, and commitment openings. Circuit obliviousness ensures that proof shape depends only on public capacities and parameters. Thus, the verifier learns the declared metadata, the committed history root, and the validity of the certified statements, but not the private records or private intermediate states.

B Design Space and Cost Derivations

This appendix has three roles: (i) it walks the plaintext Count-Min update inside ZK and quantifies the per-cost-axis pains on the running example (the *warm-up*, formerly §4.1 in earlier drafts); (ii) it names and quantifies the `naive-zkSketch` strawman as the cost to beat (formerly §4.2); and (iii) it places committed sketches relative to exact ZK indexes through a compact cost comparison.

Warm-up: plaintext Count-Min inside ZK.. Count-Min's plaintext update is a one-liner on a record (k, v): compute d keyed hashes $h_i(k)$ and increment d cells $S[i, h_i(k)]$ += v. Inside a ZK circuit this becomes a sequence of d in-circuit Poseidon evaluations ($\sim$ 300 constraints each), d private-address Merkle paths ($\sim 2 \log w \times 300 \approx 4{,}800$ constraints per record at $\log w = 8$), within-batch collision sequencing, and a state recommit. On a production deployment ($m = 1024$, $d = 7$, $w = 2^{16}$, $H \approx 300$) the direct encoding costs $\sim 3.4 \times 10^7$ constraints per batch — an order of magnitude above the optimised pipeline of §4 + §5. Three challenges crystallise the gap: **C1** private random writes at hash-determined addresses; **C2** within-batch collisions resolved in data-dependent order; **C3** capacity-dependent leakage (touched-cell count is a function of key skew). The framework's delta certificate, HPP, and capacity-bounded padding address C1, C2, and C3 respectively.

Strawman: `naive-zkSketch` *and the cost to beat.* We call the direct encoding above `naive-zkSketch`. It pays for three things the actual sketch update does not need to certify: per-record per-row Merkle paths ($\Theta(md)$), in-circuit Poseidon ($\Theta(mdH)$), and full-state commitment recompute ($\Theta(s)$). The delta-certificate construction of §5 replaces all three: the prover groups writes outside the circuit and certifies the grouping by permutation/sortedness/segment aggregation ($\Theta(md)$ rows verified, not paths opened); HPP (§**??**) replaces in-circuit Poseidon with committed lookups ($\Theta(mdH) \to \Theta(mdL)$, $L \ll H$); the homomorphic-additive vector commitment reduces the recommit to the touched cells ($\Theta(T)$ instead of $\Theta(s)$). For the running parameters with $T = 64$ and $L \approx 30$, the post-optimisation per-batch cost is around 4×10^5 constraints — an $\sim 80\times$ saving over `naive-zkSketch`. Two further circuit-level optimisations from §**??** stack on top when their preconditions hold: wide-output Poseidon2 (§**??**) cuts the in-circuit hash invocations under NativeHash from md to m, a further $d\times$ reduction on the hash share when LookupHash is not chosen; and batch deduplication (§**??**) cuts those m invocations to u unique-key permutations under skewed workloads (4–8× on Zipf-$\alpha = 1$ streams). The multiset-equality routing alternative (§**??**) changes the column-family count rather than the asymptotic, so it does not move the headline 80× but shifts constants the planner of §**??** can exploit. We do not implement `naive-zkSketch` as an evaluation column in §6; it is a rhetorical cost benchmark that makes the framework's structural savings explicit.

Comparative cost analysis. The rest of this appendix places committed sketches relative to exact ZK indexes through a compact cost comparison in PLONKish constraint counts and derives every entry of Table 1 from a per-row PLONKish accounting.

Setting. A committed dataset D of n records evolves through batches $B_1, B_2, \ldots$ of m records each. We compare an *exact index* (sorted-index commitment with lookup arguments, in the methodology of PoneglyphDB and ZKSQL [17, 22]) against a *committed sketch* maintained by the delta-certified design of §4. Each strategy admits a naive PLONKish encoding (replay the plaintext computation) and an optimized one. H denotes per-Poseidon-evaluation cost; L denotes per-lookup-argument cost.

Table 1: Asymptotic PLONKish cost per query / per single-record update for naive and optimized baselines, ignoring lower-order constants. Per-row derivations and per-family naive baselines for sketches other than CM appear in Appendix C. Boldface entries highlight where each baseline wins.

	Naive		Optimized	
Operation	Index	Sketch	Index	Sketch
Point query	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(L)$	$\Theta(dL)$
Range freq.	$\Theta(RH \log n)$	$\Theta(dH \log w)$	$\Theta(RL)$	$\Theta(dL)$
Top-k	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Distinct	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Quantile	$\Theta(nH \log n)$	$\Theta(s'H)$	$\Theta(nL)$	$\mathbf{\Theta(s'L)}$
Per-rec. update	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(H \log n)$	$\mathbf{\Theta(d+L)}$

Three observations carry the design space. *(1) Point queries are comparable across optimized baselines*: optimized index pays one lookup; optimized sketch pays d openings. *(2) Aggregate queries asymptotically favor the optimized sketch*: top-k, distinct, quantile, arbitrary-range frequency all run in $\Theta(s \cdot L)$ or $\Theta(s' \cdot L)$ vs. $\Theta(n \cdot L)$ for the index. *(3) Sketches do not universally dominate*: exact-equality tests and per-record retrieval are the index's authority.

Naive PLONKish baselines. Naive exact index. Per-record update is a $\log n$-deep Merkle path with one Poseidon per level: $\Theta(H \log n)$. Per point query: $\Theta(H \log n)$. Per range-frequency of R keys: $\Theta(RH \log n)$. Per top-k/distinct/quantile: $\Theta(nH \log n)$. *Naive sketch (CM).* Per record: d in-circuit hashes ($\Theta(dH)$) + d Merkle openings ($\Theta(dH \log w)$) + d rebindings + collision sequencing. Total: $\Theta(dH \log w)$. Aggregate decode against the committed sketch state: $\Theta(sH)$, independent of n.

Optimized PLONKish baselines. Optimized exact index. Per point query: one lookup ($\Theta(L)$); per range: $\Theta(RL)$; per top-k/distinct/quantile: $\Theta(nL)$. Per-record update under authenticated balanced-tree maintenance: $\Theta(H \log n)$. *Optimized sketch.* Delta-certified maintenance gives per-record update $\Theta(d + L)$ amortized; aggregate queries cost $\Theta(sL)$ or $\Theta(s'L)$.

Streaming regime. The per-record update ratio between optimized exact index and optimized sketch is $\Theta(H \log n/(d+L))$, which grows with n for any fixed d and reasonable H, L. At $H \approx 200, L \approx 5$, $d = 7, n = 2^{30}$, the ratio is ≈ 500. The streaming sketch-maintenance regime separates the two strategies on every per-update workload regardless of query mix.

C Naive Baselines by Sketch Family

The naive PLONKish baseline of Appendix B uses CM as the canonical example. The other families pay differently. *CM*: $\Theta(dH \log(dw))$ per record. *HLL*: naive update is one hash + one max-rebind ($\Theta(H \log s)$), but decode scans every register ($\Theta(sH)$). *SS*: per record $\Theta(sH)$ for membership probe + argmin scan + branch + evict/insert, with data-dependent control flow. *GK*: per record $\Theta(s'H)$ for sorted-position search + insertion shifts + periodic compression. The optimized delta-certified design of §4 collapses these heterogeneous naive baselines into one uniform certificate-verification pattern.

D Framework Interface and Slot Catalog

Interface tuple. A sketch is integrated by specifying $\Pi_\tau = (\Sigma_S, \Sigma_\Delta, \oplus, \text{Patch}, \text{Deco}$ together with PLONKish realizations. Σ_S: committed layout of S_t. Σ_Δ: per-record certificate row shape. $\oplus$: bounded-depth aggregation/merge operator over delta rows targeting the same address (Pattern A) or shard (Pattern B). Patch: sparse cell rebinding (Pattern A) or whole-state replacement (Pattern B). Decode$_q$: query rule. Merge: combine compatible committed states. Compat: public predicates two states must satisfy to be merge-compatible.

Table 2: L3 slot catalog. The planner selects a realization per slot at compile time. H = per-Poseidon cost; L = per-lookup cost; T = touched-cell count; $|S|$ = state size; N = HPP table size.

Slot	Realizations	Prover	Verifier
HashProfile	NativeHash (Poseidon / Poseidon2 / Anemoi / Monolith)	$\Theta(H)$	$O(1)$
	LookupHash (Plookup / LogUp / CQ)	$O(\log N)$	$\Theta(N)$ once
	HashCert (external Merkle)	$\Theta(\log N)$	$\Theta(\log N)$
SegSum / SegMax	Local recurrence	$\Theta(T)$	$O(1)$
	Prefix-difference	$\Theta(T)$	$O(1)$
Patch	SparseAddPatch + MSMConsistency (A linear)	$\Theta(T)$	$O(1)$
	SparseMaxPatch + IPA/KZG (A max)	$\Theta(T + \log \lvert S\rvert)$	$\Theta(\log \lvert S\rvert)$
	ReplaceState whole-state recommit (B)	$\Theta(\lvert S\rvert)$	$O(1)$
	Lookup-argument-backed table	$\Theta(T)$ lookups	$\Theta(\lvert S\rvert)$ once
TopS / FuseTuple	Threshold-residual certificate (TopS)	$\Theta(s)$	$O(1)$
	$\log s'$ pairwise rounds (FuseTuple)	$\Theta(s' \log s')$	$O(1)$
Capacity	Static / Adaptive / DP-via-CCS	varies	varies

Compiler. The deployment-aware sketch compiler is presented as Algorithm **??** in §**??**; per-sketch planner outputs across the three deployment profiles are tabulated in Table **??** of the same subsection.

E Detailed Gate Specifications

This appendix gives witness/output/verification-target/realization specifications for the gates introduced or adapted by zkSketch, in the format used by PoneglyphDB and VPJoin [2, 17]. Established primitives (RangeCheck, IsZero, IsEq, Cmp, Mux, MinMax,

ArgMin, ArgMax, PrefixSum, PrefixMax, Permute, LexSort, ActiveFirst, Segment, SegSum, SegMax, StableCompact, Disjoint) follow their canonical realizations and are not detailed here.

E.1 Canonical Table Encodings

All sketch gates use a small number of table encodings. We state them explicitly because most soundness bugs in PLONKish database circuits arise from missing active flags, dummy rows, or permutation columns. *Active rows and dummy rows.* Every witness table has an active bit a_j. The circuit enforces $a_j(a_j - 1) = 0$. For a tuple $t_j = (a_j, x_{j,1}, \ldots, x_{j,r})$, inactive rows must equal a fixed dummy: $(1 - a_j)(x_{j,\ell} - \perp_\ell) = 0$ for every field ℓ. *Compressed tuple encoding.* For permutation checks, a row is compressed as $E_j = tag_j + \alpha_0 a_j + \alpha_1 x_{j,1} + \cdots + \alpha_r x_{j,r}$ where the α_i are verifier challenges; tags distinguish tables sharing columns. The circuit uses a permutation or grand-product argument to prove multiset equality of encoded rows. *Sorted tuple keys.* Lexicographic order on a multi-column key is enforced by equality-prefix bits $p_{j,\ell} = p_{j,\ell-1} \cdot [k_{j,\ell} = k_{j+1,\ell}]$ and less-than bits $q_{j,\ell} = [k_{j,\ell} \le k_{j+1,\ell}]$, with the order condition $\sum_\ell p_{j,\ell-1}(1 - [k_{j,\ell} = k_{j+1,\ell}])q_{j,\ell} + p_{j,r} = 1$. In practice the key may be packed into a bounded integer when its packed range fits the field, replacing the multi-column comparison with one range check on the difference. Realized in Halo2 as advice columns plus permutation-argument checks; the equality-prefix and less-than bits are degree-2 custom gates [39].

E.2 Commitment Scheme Choice

A naive Merkle-tree commitment costs $\Theta(\log |S| \cdot H)$ per cell access, dominating the update circuit when T is in the thousands. The framework recommends:

Table 3: Recommended commitment schemes per pattern.

Pattern	Sketches	Scheme	Per-batch cost
A linear	CM	Homomorphic-additive vector commitment	$O(T) + O(1)$
A max	HLL	Vector commitment + batched IPA/KZG	$O(T + \log \lvert S \rvert)$
B	SS, GK	Vector commitment over bounded state	$O(\lvert S \rvert)$ MSM
Any (medium)	state $\le 2^{20}$	Lookup-argument-backed table	$O(T)$ lookups

E.3 TopS

TopS$(G, s, \prec, N, R)$. **Witness.** Grouped table G, kept table N of capacity s, residual table R, public total order $\prec$. **Verification target.** N contains the s best active rows of G under $\prec$. **Realization.** (i) Conservation $G \equiv N \cup R$ via Permute; (ii) N sorted by $\prec$; (iii) every active residual $r \in R$ no better than the last active row of N (or no better than a dummy threshold), checked by Cmp against a witness threshold τ. Realized in Halo2 as one permutation argument over compressed-tuple advice columns plus per-residual range-check on the Cmp difference; circuit cost is linear in $|G| + |N| + |R|$, which is asymptotically tighter than any oblivious top-s sorting network [39].

E.4 HashProfile

HashProfile(P, i, k). Returns $b \in [0, w)$ with $b = h_i(k)$ under public profile digest P. Three modes: *NativeHash* (in-circuit ZK-friendly hash), *LookupHash* (membership of (P, i, k, b) in a committed profile table [12, 39]), *HashCert* (external Merkle opening or universal-hash certificate). The HLL LeadingRun variant is detailed in Appendix F.

Three-mode cost profile (HPP).. HPP commits the hash-evaluation table $T_H = \{(P, i, k, h_i(k))\}$ of size N as a public commitment C_H, then verifies per-record hashes by lookup. The three HashProfile modes have different cost profiles. *NativeHash* pays $\Theta(H)$ per record on the prover and $O(1)$ on the verifier; the table C_H is unused. *LookupHash* pays $O(\log N)$ amortized per record on the prover via a Plookup or LogUp lookup against C_H; the verifier pays $\Theta(N)$ once per profile-table commit. *HashCert* offloads hashing to an external certificate (Merkle opening or universal-hash witness), raising verifier cost but minimizing prover cost on long-lived deployments. The deployment chooses: HashCert is most useful when the same profile is reused across many independently maintained sketches; LookupHash is most useful when batch sizes are large and the profile table fits within Halo2 lookup capacity; NativeHash is the fall-back for ad-hoc deployments. Realized in Halo2 as a Plookup/LogUp argument over advice columns [12, 39]; the per-batch hash share collapses from $\Theta(md \cdot H)$ to $\Theta(m \cdot L)$ under LookupHash, with the $\Theta(N \cdot H)$ one-time preprocessing cost amortized across all subsequent batches under the same profile.

E.5 MSMConsistency

Setting. Fix commitment key $\{G_\kappa\}_{\kappa \in [N]}$ for a vector commitment of capacity $N = |S|$. **MSMConsistency**(A, C_Δ)**.** Witness: aggregate $A = \{(\kappa_j, \Delta_j)\}$ of capacity $T_{\max}$ with active flags. Verification: $C_\Delta = \sum_j a_j \Delta_j G_{\kappa_j}$. Realization in three phases: (i) bucket-key extraction with RangeCheck $\kappa_j < N$ and Mux for $\hat{\Delta}_j = a_j \Delta_j$; (ii) per-row scalar multiplication $P_j = \hat{\Delta}_j G_{\kappa_j}$ via lookup-table key access plus windowed scalar multiplication ($\sim$ 100–200 constraints per scalar under Pasta-cycle Halo2 IPA); (iii) running-sum $Q_j = Q_{j-1} + P_j$ with $Q_{T_{\max}} = C_\Delta$ as a curve-equation check. Cost: $\Theta(T_{\max})$ scalar mults + EC adds, independent of $|S|$.

E.6 State, Commitment, and Tree-Append Gates

OpenCell$(C, \kappa, x, \pi_{open})$. Certifies $C[\kappa] = x$. Under homomorphic-additive vector commitment, implicit in SparseAddPatch's group-relation check; under batched IPA/KZG, amortized into one multi-opening of cost $O(T + \log |S|)$; under Merkle (naive baseline only), depth-$\log |S|$ path; under lookup table, single lookup.

SparseAddPatch(C, C', A). Verifies $C'[\kappa] = C[\kappa] + \Delta$ for $(\kappa, \Delta) \in A$. Under the recommended homomorphic-additive vector commitment, the gate verifies $C' = C + \mathsf{Com}(\Delta)$ together with MSMConsistency tying $\mathsf{Com}(\Delta)$ to A; *no per-cell Merkle path required.*

SparseMaxPatch(C, C', A). Same as SparseAddPatch but $x' = \max(x, \Delta)$. The homomorphic shortcut does not apply; recommended realization is a batched IPA/KZG multi-opening plus in-circuit max gates.

ReplaceState(C, C', T, T'). Whole-state replacement under bounded-state vector commitment of Pattern B; recommit cost is one MSM of size $|T'| \le s$ or s'.

MergeTree-Append$(R, R', t, \mathsf{Com}(S_t), \pi_t, M)$. The composition-extension gate for the binary-tree topology (§4.3): enforces that R' is the binary-tree root obtained by appending a leaf at position t carrying $(\mathsf{Com}(S_t), \pi_t)$ to the tree with root R. For merge proofs across shards, the tree-append takes the children's roots as siblings instead of a single appended leaf. Prevents replay, reordering, skipped batches, double-counting, hash-profile substitution. The analogous gates for the other topologies are *Chained-Fold* (§4.3: extends the accumulator A_{t-1} by folding in the new commitments $\mathsf{Com}(S_t), \pi_t$, producing A_t and the implicit root $R^{(t)} = \mathsf{hash}(A_t)$) and *k-ary-Merge* (§4.3: takes k sibling commitments and produces their composed root). Each variant preserves the same soundness profile: replay / reorder / drop attacks change the root commitment except with negligible probability.

Compat. Public-metadata predicates: sketch type, dimensions, key domain, hash/sign profile, counter representation, group schema, deletion policy, release policy, overflow bounds. Merge gates additionally check disjoint partition coverage.

F HyperLogLog (Pattern A specialization)

HyperLogLog (HLL) is a Pattern A sketch with a max-based update operator. The committed state is an array of s small registers

$$S_{\mathsf{HLL}}[b] \in \{0, \dots, \ell\}, \qquad b \in [s].$$

Each uploaded record is a key k. The declared hash profile contains two hash functions:

$$h_1 : \mathcal{K} \to [s], \qquad h_2 : \mathcal{K} \to \{0,1\}^{\ell}.$$

For a key k, the plaintext update computes

$$b = h_1(k), \qquad \rho = \mathsf{LeadingOnes}(h_2(k)),$$

and updates the register by

$$S_{\mathsf{HLL}}[b] \leftarrow \max\{S_{\mathsf{HLL}}[b], \rho\}.$$

The exact convention for LeadingOnes, including whether it is zero- or one-indexed and how all-one strings are handled, is fixed by the public metadata M.

F.1 Update Circuit

The HLL transition circuit certifies one committed epoch

$$B_t = \{k_j\}_{j=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. The batch effect is a sparse register-wise maximum patch: one row per touched register, carrying the largest ρ-value contributed by the epoch to that register.

HyperLogLog Update Circuit.
Public input. Batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing s, ℓ, the hash profile, register representation, estimator convention, commitment scheme, and public capacities.

Private witness. Batch table B, generated-register table U, register-sorted table $U^{\uparrow}$, sparse max-patch table A, and old/new opening tables O, N.
Output. A proof that C_{new} commits to the result of applying the declared HLL epoch update to C_{old} using the committed batch C_{B_t}.
Verification target. For every register $b \in [s]$,

$$S_t[b] = \max\left(S_{t-1}[b], \max_{j:\, h_1(k_j)=b} \mathsf{LeadingOnes}(h_2(k_j))\right).$$

Step 1: Bind the private batch to the public commitment. The circuit starts with a private batch table

$$B[j] = (a_j, k_j),$$

where $a_j \in \{0,1\}$ is an active flag. It verifies that active rows of B open from C_{B_t}, and that inactive rows are canonical dummy rows.

Step 2: Generate register contributions. For each active row $B[j] = (1, k_j)$, the circuit computes

$$b_j = h_1(k_j), \qquad y_j = h_2(k_j), \qquad \rho_j = \mathsf{LeadingOnes}(y_j),$$

and checks

$$0 \le b_j < s, \qquad 0 \le \rho_j \le \ell.$$

The generated-register table is

$$U[j] = (a_j, b_j, \rho_j).$$

Inactive batch rows produce canonical dummy rows in U.

Step 3: Bring contributions to the same register together. The prover supplies a register-sorted table $U^{\uparrow}$. The circuit verifies

$$U^{\uparrow} \equiv U$$

as multisets over the semantic columns (a, b, ρ), and checks that $U^{\uparrow}$ is sorted by register id b, with inactive rows placed last. After this check, all contributions to the same HLL register form one contiguous segment.

Step 4: Aggregate by register using max. The circuit performs segment aggregation over $U^{\uparrow}$. For each active segment with register id b, it computes

$$M_b = \max_{j:\, h_1(k_j)=b} \rho_j.$$

The output is a sparse max-patch table

$$A[r] = (\beta_r, b_r, M_{b_r}),$$

where $\beta_r \in \{0,1\}$ is an active flag. Each active row of A corresponds to one HLL register touched by the epoch.

Step 5: Apply the sparse max patch. The prover supplies old and new opening tables O and N for the registers listed in A. For every active patch row $(b, M_b) \in A$, the circuit checks

$$N[b] = \max\{O[b], M_b\}.$$

It also verifies that $O[b]$ opens correctly against C_{old}, that $N[b]$ opens correctly against C_{new}, and that all opened register values are in the declared range.

Unlike Count-Min, HLL cannot use an additive commitment relation of the form $C_{\text{new}} = C_{\text{old}} + \mathsf{Com}(\Delta)$, because the update operator is max rather than addition. The state-rebinding check therefore uses the mechanism supported by the chosen commitment scheme: a sparse update proof for the touched coordinates, batched old/new openings together with an equality-outside-the-patch proof, or a whole-state replacement proof when sparse max updates are not supported.

Step 6: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that M fixes the HLL parameters, hash profile, register representation, estimator convention, commitment scheme, and public capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Update summary. The HLL transition circuit proves four claims: the batch rows are bound to C_{B_t}; each active key produces the declared register contribution $(h_1(k), \mathsf{LeadingOnes}(h_2(k)))$; contributions to the same register are aggregated by maximum; and the new committed state is obtained by applying the resulting sparse max patch to the old committed state.

F.2 Query Circuit

An HLL query is parameter-free: it estimates the number of distinct keys from the committed register array. In zkSketch, queries are answered against a closed sketch version, so the query circuit reads only the committed HLL state.

The prover opens the register array

$$S_t[1], \ldots, S_t[s]$$

from the latest state commitment. The circuit verifies the openings and then computes the public statistic required by the declared estimator. There are two supported modes.

HLL estimator mode. The circuit computes

$$Z = \sum_{b=1}^{s} 2^{-S_t[b]},$$

using the fixed-point representation declared in M, and checks

$$ans = \alpha_s s^2 Z^{-1},$$

or the deterministically rounded version specified by the release policy. The constant α_s, fixed-point precision, inverse check, and rounding rule are public metadata.

LogLog score mode. For backends where exponentiation and inversion are expensive, the circuit can instead certify the LogLog-style score

$$R = \sum_{b=1}^{s} S_t[b].$$

The public release policy then maps R to the reported cardinality estimate, for example by a deterministic formula of the form

$$\widehat{D} = \alpha'_s s\, 2^{R/s}.$$

Since the post-processing formula is public and deterministic, the release circuit can certify it separately when the deployment publishes $\widehat{D}$ rather than R.

HyperLogLog Query Circuit.
Public input. State commitment C_{state}, metadata M, and public answer *ans* or public score R.

Private witness. Opened register array $S_t[1], \ldots, S_t[s]$, opening witnesses, and the intermediate estimator values required by the selected estimator mode.
Verification target. The opened registers are the committed HLL state, and the public output is the declared estimator or score computed from those registers.
Checks. The circuit checks state openings, register range constraints, the declared summation or estimator relation, and the deterministic rounding or release rule when included in the query proof.

G SpaceSaving (Pattern B specialization)

We instantiate SpaceSaving as the bounded count-based summary used by the streaming sketch framework. The committed state stores at most s active key–count pairs

$$S_{\mathrm{SS}}[r] = (a_r, k_r, c_r), \qquad r \in [s],$$

where $a_r \in \{0, 1\}$ is an active flag. Active rows are sorted by a public total order

$$(c, k) \succ (c', k')$$

that orders larger counts first and breaks ties deterministically by key. Inactive rows are canonical dummies and appear after active rows.

The declared epoch update follows the batch merge rule: union the old summary rows with the new epoch rows, group by key and sum counts, then retain the top s key–count rows under the public order. This is a Pattern B certificate because the update is a global summary replacement, not a sparse update to fixed cells.

G.1 Update Circuit

The SpaceSaving transition circuit certifies one committed epoch

$$B_t = \{k_j\}_{j=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. If the workload uses weighted updates, each batch row is (k_j, v_j); the unweighted case is recovered by setting $v_j = 1$.

SpaceSaving Update Circuit.
Public input. Batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing the capacity s, key representation, count representation, tie-breaking order, commitment scheme, and public table capacities.
Private witness. Batch table B, old summary table S_{old}, union table U, key-sorted union table $U^{\uparrow k}$, grouped candidate table G, kept table S_{new}, residual table R, and commitment-opening witnesses.
Output. A proof that C_{new} commits to the result of applying the declared SpaceSaving epoch update to C_{old} using the committed batch C_{B_t}.
Verification target. Let

$$W_k = \sum_{r:\, S_{\mathrm{old}}[r].k = k} S_{\mathrm{old}}[r].c + \sum_{j:\, k_j = k} v_j.$$

The new committed state contains the top s active key–count pairs (k, W_k) under the declared count/key order, padded with canonical dummy rows if fewer than s candidates exist.

Step 1: Bind the old state and the private batch. The circuit verifies that the private old summary table

$$S_{\mathrm{old}}[r] = (a_r, k_r, c_r)$$

opens from C_{old}, is sorted by the declared count/key order, has distinct active keys, and uses canonical dummy rows after the active prefix. It also verifies that the private batch table

$$B[j] = (a_j, k_j, v_j)$$

opens from C_{B_t}, with $v_j = 1$ in the unweighted setting and canonical dummy rows for inactive entries.

Step 2: Form the union table. The prover supplies a union table U containing all active old summary rows and all active batch rows:

$$U = \{(k_r, c_r, \mathsf{old}) : S_{\mathrm{old}}[r] \text{ active}\} \cup \{(k_j, v_j, \mathsf{new}) : B[j] \text{ active}\}.$$

The circuit verifies that U is exactly this multiset union, including active flags and source tags. Source tags are not part of the logical summary; they are only used to bind rows to either the old state or the new committed epoch.

Step 3: Group by key and sum counts. The prover supplies a key-sorted table $U^{\uparrow k}$. The circuit checks

$$U^{\uparrow k} \equiv U$$

as multisets and verifies that $U^{\uparrow k}$ is sorted by key, with inactive rows last. It then checks segment aggregation over equal-key runs, producing a grouped candidate table

$$G[p] = (\gamma_p, k_p, W_p),$$

where

$$W_p = \sum_{r:\, S_{\mathrm{old}}[r].k = k_p} S_{\mathrm{old}}[r].c + \sum_{j:\, k_j = k_p} v_j.$$

Thus G contains one active row per distinct key appearing either in the old summary or in the new epoch.

Step 4: Certify top-s retention. The prover supplies two tables: a kept table S_{new} of capacity s, and a residual table R. The circuit verifies the conservation relation

$$G \equiv S_{\mathrm{new}} \uplus R$$

as multisets over active flags, keys, and counts. It verifies that S_{new} is sorted by the declared count/key order, has distinct active keys, and is padded with canonical dummy rows.

Let τ be the last active row of S_{new}, or a public dummy threshold if fewer than s candidates exist. The circuit checks the threshold-residual condition

$$R[q] \preceq \tau$$

for every active residual row $R[q]$, where $\preceq$ is the non-strict version of the declared order. This proves that no omitted candidate is better than the worst retained candidate. Therefore, the kept table contains exactly the top s grouped candidates.

Step 5: Recommit the bounded summary. The circuit verifies that the kept table opens to the new state commitment C_{new}. This is a whole-state replacement check: the SpaceSaving state is a bounded ordered summary, and its correctness is a global top-s property rather than a per-cell patch.

Step 6: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that M fixes the declared SpaceSaving variant, capacity s, key representation, count representation, ordering rule, commitment scheme, and public table capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Update summary. The SpaceSaving transition circuit proves four claims: the old summary and epoch rows are bound to their public commitments; the candidate table correctly groups old and new contributions by key; the new summary contains the top s grouped candidates under the declared order; and the new state commitment binds to this kept table. This is the Pattern B analogue of batch-effect certification: the certificate proves a valid bounded summary replacement rather than a sparse numerical patch.

G.2 Query Circuit

A SpaceSaving top-c query returns the first c active rows of the committed summary under the declared order. Since zkSketch answers queries against closed sketch versions, the query circuit reads only the committed state S_t. If a deployment keeps an explicit unmerged buffer, the buffer must be merged into the summary before answering a top-c query.

SpaceSaving Top-c Query Circuit.
Public input. State commitment C_{state}, query size c, metadata M, and public answer table Ans.
Private witness. Opened SpaceSaving table S_t and opening witnesses.
Verification target. The opened table is the committed SpaceSaving state, is sorted by the declared count/key order, and

$$Ans[r] = S_t[r], \qquad r = 1, \ldots, c,$$

with canonical dummy outputs if fewer than c active rows exist.
Checks. The circuit checks the state opening, active-prefix structure, distinct-key condition, sortedness by the declared order, and equality between the public answer table and the first c rows of the opened state.

Point-frequency query. A point query for a key q can be supported as an auxiliary decoder. The prover opens the committed SpaceSaving table and supplies equality bits

$$e_r = [S_t[r].k = q].$$

The circuit checks $e_r \in \{0, 1\}$, verifies the equality relation for each row, and computes

$$ans = \sum_{r=1}^{s} e_r \cdot S_t[r].c.$$

Because the update circuit enforces distinct active keys, at most one row contributes. If no row matches q, the answer is 0. This is the declared SpaceSaving estimate for q under the bounded summary.

H Constraint-Count Breakdown

We use the representative deployment $m = 1024$, $d = 7$, $w = 2^{16}$, $T \approx 7000$, $s, s' \le 2^{14}$, Halo2 IPA over Pasta cycles. The numbers are order-of-magnitude estimates intended to ground relative shares.

Commitment-scheme cost across regimes. Naive Merkle: $\sim T \cdot 2850 \approx 2 \times 10^7$ constraints—larger than every other line item combined. *Homomorphic-additive vector commitment* (Pattern A linear): $T \cdot 150 + O(1) \approx 1.05 \times 10^6$, a 15–30× improvement. *Batched IPA/KZG* (Pattern A max, HLL): $T + \log |S| \approx 7019$ multi-opening ops, in the 10^5–10^6 range. *Whole-state recommit* (Pattern B): one MSM of size $|S| \le s$ or s'.

Per-family closed-form cost equations. For Count-Min (Pattern A linear under HPP-LookupHash + homomorphic-additive commitment) the per-batch update cost is, as in §??,

$$C_{\text{prv}}^{\text{CM}} = m \cdot L_{\text{lkp}} + 2(md) \log(md) \cdot c_{\text{cmp}} + T \cdot (c_{\text{seg}} + c_{\text{patch}}) + c_{\text{lin}}.$$

For HyperLogLog (Pattern A max under batched IPA/KZG opening),

$$C_{\text{prv}}^{\text{HLL}} = m(L_{\text{lkp}} + c_{\text{lr}}) + 2m \log m \, c_{\text{cmp}} + T c_{\text{seg}} + (T + \log |S|) c_{\text{open}} + c_{\text{lin}},$$

where c_{lr} is the per-record LeadingRun gate constant and c_{open} is the batched-multi-opening per-row constant. For SpaceSaving (Pattern B with whole-state recommit on a state of bounded size s),

$$C_{\text{prv}}^{\text{SS}} = (s+m)c_{\text{union}} + 2(s+m) \log(s+m) c_{\text{cmp}} + (s+m)c_{\text{seg}} + s c_{\text{topS}} + s c_{\text{patch}} + c_{\text{lin}},$$

where c_{union} is the per-row SS-Union copy constant and c_{topS} is the threshold-residual per-row constant. For Greenwald–Khanna (Pattern B with $\log s'$ FuseTuple rounds),

$$C_{\text{prv}}^{\text{GK}} = (s'+m)c_{\text{insort}} + (s'+m) \log(s'+m) c_{\text{cmp}} + (s'+m) \log(s') c_{\text{fuse}} + s' c_{\text{patch}} + c_{\text{lin}},$$

where c_{insort} is the GK-InsertSort per-row constant, c_{fuse} is the FuseTuple per-pairwise-decision constant, and the $\log s'$ factor reflects the rounds of pairwise fusion (limit F1 of §??). The constants $\{c_{\text{cmp}}, c_{\text{seg}}, c_{\text{patch}}, c_{\text{lin}}, L_{\text{lkp}}, c_{\text{lr}}, c_{\text{open}}, c_{\text{union}}, c_{\text{topS}}, c_{\text{insort}}, c_{\text{fuse}}\}$ are calibrated against published Halo2 cost microbenchmarks [39] and yield the per-family numbers in Tables 4–6 below.

Table 4: Count-Min update with HPP and homomorphic-additive vector commitment.

Component	Constraints	Share
Hash verification (HPP lookup, $md = 7168$)	36,000	2%
Sort / permutation	750,000	40%
Segment aggregation	70,000	4%
Commitment patch	1,050,000	55%
Tree append	1,000	<1%
Total	$\sim 1.9 \times 10^6$	100%

Table 5: HyperLogLog update with batched IPA opening.

Component	Constraints	Share
Hash verification (m Poseidon + LeadingRun)	154,000	27%
Sort / permutation	100,000	18%
Segment max + sparse rebind	10,000	2%
Commitment opening	300,000	53%
Total	$\sim 5.6 \times 10^5$	100%

After applying the recommended schemes, no single component dominates by an order of magnitude; total cost lands in the 0.5–3.0 million constraint range. This is a 10–20× improvement over the naive Merkle baseline.

Table 6: SpaceSaving update with whole-state recommit ($s = m = 1024$).

Component	Constraints	Share
Buffer union (SS-Union)	30,000	6%
Sort + segment grouping	225,000	44%
Top-s certificate	100,000	20%
ReplaceState (MSM over $\|S\|$)	155,000	30%
Total	$\sim 5.1 \times 10^5$	100%

Table 7: Greenwald–Khanna update with $\log s'$ fusion rounds and whole-state recommit ($s' \approx 9.8 \times 10^4$, $m = 1024$).

Component	Constraints	Share
GK-InsertSort + InitDelta	350,000	23%
FuseTuple ($\lceil \log s' \rceil = 17$ rounds)	720,000	48%
StableCompact	80,000	5%
ReplaceState (MSM over $\|S\|$)	350,000	23%
Total	$\sim 1.5 \times 10^6$	100%

I Capacity Co-Sketch (CCS): DP-Bounded Capacity Privacy

zkSketch's three body-level compositional techniques — SDC, HPP, NDC — are engineering optimisations on the basic delta certificate that reduce cost without changing what leaks. The capacity co-sketch (CCS) is qualitatively different, because it changes *what leaks*: it converts the choice of public per-batch capacity from a deterministic, data-dependent quantity into a differentially private one. We describe the construction in detail here; the privacy-composition argument is then Proposition **??** in Appendix J.

Bottleneck. The padded capacities $T_{\max}$, s, s', G are public, and circuit obliviousness depends on this. If a deployment fixes capacities statically at worst-case values, padding overhead is large. If it instead adapts the capacity per batch to minimise padding, the choice of capacity itself leaks data-dependent quantities: touched-cell counts correlate with key skew, distinct-key cardinality with workload concentration, and so on.

Insight. Replace the deterministic adaptive capacity with a differentially private one. The next-batch capacity is then a DP-bounded function of the touched-cell history, and the public circuit shape becomes a function only of DP-released capacities. Circuit obliviousness composes with a quantifiable leakage budget instead of a vague best-effort hope, and the leakage joins the cost surface (§**??**) as a third coordinate alongside prover and verifier work. This lifts VPJoin's DP-guided capacity idea [2] from join-intermediate cardinalities to sketch touched-cell cardinalities.

Construction. CCS maintains a small auxiliary committed sketch $\widetilde{T}_t$ alongside the primary sketch. $\widetilde{T}_t$ tracks an estimator of the touched-cell cardinality of recent batches. The next batch's padded capacity is

$$T_{\max}^{(t+1)} = \mathrm{ceil}\Big(\widetilde{T}_t + \mathrm{Lap}\big(\tfrac{\Delta_T}{\varepsilon}\big)\Big),$$

where $\mathrm{Lap}(\Delta_T/\varepsilon)$ is Laplace noise calibrated to the per-record contribution sensitivity Δ_T and the per-batch privacy budget ε. The sensitivity is the maximum number of distinct cells a single record can touch: $\Delta_T = d$ for Count-Min (one cell per row), $\Delta_T = 1$ for HyperLogLog (one register per record), $\Delta_T = 1$ for SpaceSaving (at most one promotion per record), and $\Delta_T = 2$ for Greenwald–Khanna (at most two adjacent rank intervals per record). To stop the prover from forging $\widetilde{T}_t$ adversarially, the CCS update gate binds the primary sketch and the auxiliary sketch with a coupling permutation: the touched-cell aggregate A used to update the primary must be the very same A used to update $\widetilde{T}_t$.

Savings. CCS converts an uncountable leak (an exact data-dependent capacity release) into a quantifiable one ($(\varepsilon_N, \delta_N)$-DP) at modest cost. The auxiliary sketch $\widetilde{T}_t$ is a Count-Min counter of dimensions $(d_{\mathrm{cc}}, w_{\mathrm{cc}}) = (3, 256)$ tracking touched-cell cardinality, with state size $|\widetilde{T}_t| = 768$ counters — about 0.2% of a representative primary CM state ($d = 7$, $w = 2^{16}$). The CCS update circuit adds an extra HashProfile evaluation per touched cell, an extra SegSum, and the coupling permutation; on the running CM deployment this contributes ∼15K constraints, or ∼1% over the primary update. Composing N such per-batch capacity releases gives $(\varepsilon_N, \delta_N)$-differential privacy under standard composition (Proposition **??**), with advanced-composition tightening when N is large. The planner of §**??** treats the resulting bound as the leakage coordinate of the three-dimensional cost surface.

Soundness. The coupling permutation in the construction binds the auxiliary sketch to the same touched-cell aggregate A that the primary sketch updates against, so a prover cannot forge $\widetilde{T}_t$ adversarially without breaking the permutation argument. The DP guarantee is unconditional on prover honesty: the Laplace noise is sampled from a committed seed via the release-circuit mechanism of §3, and the seed is bound by M before any capacity is released, so the noise distribution is fixed independently of the data observed in any batch.

Capacity-policy trade-off. The three capacity policies span a continuous design space, summarised in Table 8. *Static* pads to the worst-case capacity once and forever; cost is high (the circuit always pays for the worst case) but leakage is zero (the public capacity reveals nothing about the data). *Adaptive* resizes the capacity per batch to track actual workload; cost drops by the padding ratio but every capacity release leaks an exact data-dependent quantity, so $\mathcal{L} = \infty$ under our obliviousness definition. *DP-via-CCS* sits between: the capacity is data-adaptive but DP-noised, so cost approaches adaptive in expectation while leakage is bounded by $(\varepsilon_N, \delta_N)$. The planner's choice depends on the deployment's leakage budget: regulated audit-grade deployments prefer Static (zero leakage at higher constant cost); throughput-grade deployments tolerate Adaptive when capacity itself is not sensitive (e.g., already-public touched-cell counts); leakage-tight deployments take DP-via-CCS as the principled middle ground.

J Soundness Proofs

J.1 Update Soundness (Theorem ??)

Let $\mathcal{P}^*$ be a PPT prover producing an accepted update proof for x_t. The PLONKish extractor recovers w_t satisfying every circuit constraint. We argue w_t contains the required (S, B_t, S') via four claims. *(i) Faithful opening.* The witness contains openings of C_{old} and C_{B_t}; commitment binding makes any extracted opening unique. *(ii)*

Table 8: Capacity-policy trade-off. $T_{\mathbf{wc}}$ is the worst-case touched-cell count; $T_{\mathbf{actual}}$ is the realised value under the workload; $\rho = T_{\mathbf{actual}}/T_{\mathbf{wc}}$ is the padding ratio.

Policy	Per-batch padding cost	Leakage
Static	$\Theta(T_{\text{wc}})$	0
Adaptive	$\Theta(\rho \cdot T_{\text{wc}})$	∞ (exact value released)
DP-via-CCS	$\Theta(\rho \cdot T_{\text{wc}}) + \widetilde{O}(1)$	$(\varepsilon_N, \delta_N)$-DP

Faithful delta generation. The Generate gate enforces $\Delta = \text{Generate}_\tau(B_t)$ pointwise via polynomial constraints under the declared profile. *(iii) Faithful aggregation.* Permute, LexSort, Segment, and SegSum/SegMax force the sorted delta table to be a permutation of the generated table and the per-segment aggregate to be the correct sum/max per cell. *(iv) Faithful patch and composition extension.* The patch gate forces $C_{new} = \text{Com}(A_\tau.\text{Update}(S, B_t))$ uniquely by binding. The composition-extension gate (Chained-Fold, Tree-Append, or k-ary-Merge per the chosen topology of §4.3) forces $R^{(t)}$ exactly: the new composition root differs from any honestly-extended root only if the prover violates collision resistance or the recursive-SNARK knowledge soundness of the substrate. The conjunction is exactly $\mathcal{R}^{\tau}_{\text{upd}}$.

J.2 Framework-Level Closure (Theorem J.2)

Suppose every slot of Π_τ admits a sub-soundness lemma against the certificate catalog (Table **??**). Then the update circuit instantiated for τ is knowledge-sound for $\mathcal{R}^{\tau}_{\text{upd}}$. The proof follows from per-slot sub-soundness composed via the same extraction-then-binding argument as Theorem **??**, with each slot's local invariant supplying one step of claim (ii)–(iv).

J.3 SDC Closure (Theorem J.3)

The SDC update circuit for K coupled sketches is knowledge-sound for the conjunction of K per-sketch update relations $\mathcal{R}^{\tau_k}_{\text{upd}}$, with one shared sort/permutation/tree-append and per-sketch patches. Tag-aware permutation ensures the sorted table cleanly separates the K sketches; per-sketch patch gates inherit per-sketch sub-soundness.

J.4 Workflow Soundness (Theorem ??)

The argument composes per-node soundness (per-relation extractors) with edge-commitment binding and recursive-SNARK composition-root binding (§4.3). For each edge $u \to v$ carrying boundary commitment Com, the equality of u's output and v's input commitment under the verifying check, plus binding, implies the underlying values are equal. The composition root commitment $R^{(t)}$ then forces the temporal aggregation order: any reordering of subproofs produces a different root under whichever composition topology (chained, binary-tree, or k-ary) is in use, contradicting the public-input fix and the underlying recursive-SNARK knowledge soundness. The argument is uniform across topologies because each delivers a single binding root with the same soundness profile.

J.5 CCS Privacy Composition (Proposition ??)

The CCS releases the public capacity $T^{(t+1)}_{\max} = \text{ceil}(\widetilde{T}_t + \text{Lap}(\Delta_T/\varepsilon))$ once per batch, where Δ_T bounds the per-record contribution to touched-cell count. The composition of N such per-batch releases is $(\varepsilon_N, \delta_N)$-differentially private under standard sequential composition; under advanced composition for the Laplace mechanism, ε_N grows as $O(\sqrt{N \log(1/\delta_N)} \cdot \varepsilon)$ rather than $N \cdot \varepsilon$. A deployment chooses the per-batch ε to fit a global per-audit-window budget $(\varepsilon_N, \delta_N)$; the planner of §**??** treats the resulting bound as the leakage coordinate of the three-dimensional cost surface.

K Future Extensions

The chained, binary-tree, and k-ary composition topologies of §4.3 cover the recursive-SNARK composition design space at the framework level. Two further directions remain open. *Hybrid topologies*: mixing chained folding within k-ary leaves — a batched leaf of size m runs an internal folding chain over its m subproofs, then participates in a k-ary tree — combines the per-batch latency of folding with the parallelism of trees. We leave a careful evaluation to future work. *Adaptive arity*: dynamically tuning k across tree levels to match the prover's available cores at each layer (high k near the root where parallel work is exhausted; low k at the leaves where per-merge circuits are cheap) is a natural compiler extension.